



UIT University
Department of Computer Science
(SOFTWARE ENGINEERING)
FINAL YEAR PROJECT REPORT
Batch Fall-2022

TEACHER ASSIST (OBE BASED)
Automated Teaching Assistant with Gap Analysis and OBE Content Generation

By

UBAIDULLAH	22FA-011-SE
NABEEHA AYAZ	22FA-025-SE
ALYANUDDIN	22FA-053-SE
OMAIMA IRFAN	22FA-060-SE

Supervised by

Dr. Muhammad Wasim
and
Dr. Lubaid Ahmed

*ST-13, Block 7, Gulshan-e-Iqbal, Abul Hasan Isphahani Road, Opposite Safari Park, P.O. Box 75300,
Karachi, Pakistan. Phone: 34978274-5; 34994305; 34982476; <http://www.uit.edu>*

Declaration

It is stated that this project report has been prepared by me solely for the purpose of fulfilling the requirements of the course and is my own work and that all references to work from others have been duly cited and quoted. Also, I say that it has not been applied for the purpose of obtaining any other degree or award from UIT University or any other University ever.

Syed Ubaidullah Bukhari
(22FA-011-SE)

Signature:

Nabeeha Ayaz
(22FA-025-SE)

Signature:

Alyan Uddin
(22FA-053-SE)

Signature:

Omaima Irfan
(22FA-060-SE)

Signature:

Date:

Acknowledgments

All praise and gratitude to Allah SWT, with Whose guidance, blessings and support we have the ability to take up and carry on this project.

We are thankful to Dr. Muhammad Wasim for his expert guidance, expert advice and constant encouragement throughout this project. Through their input, we've been able to determine the trajectory of our research and development and have seen how it helps us solve problems along the way.

We would also like to express our gratitude to Dr. Lubaid Ahmed for his constant help, constructive comments, and suggestions, which have helped a lot in developing our project methodology and making it better in terms of quality.

We would also like to express our special gratitude to all our faculty members, technical staff and colleagues who have shared their knowledge and resources and have supported us in the smooth implementation of this project.

This acknowledgment would be incomplete without appreciating the moral support and encouragement from our families and friends, who continue to motivate us throughout this journey.

Abstract

Teacher Assist is an AI powered Academic Automation that supports Outcome Based Education. It automates critical core academic processes to free up more time for Faculty to teach and for Administration to innovate. Faculty upload presentations, Course Information Sheets, Question papers, Mark sheets etc. and Teacher Assist intelligently processes these documents utilizing state of the art Natural Language Processing capabilities and empowers it to extract and map Course Learning Outcomes to relevant content locations in the course by specifying corresponding Cognitive Levels as per Bloom's Taxonomy. Teacher Assist then instantly and automatically generates high quality content for assignments, quizzes, Mid Term and End Term Exams, consistently embedding them in the prescribed course structure while strictly adhering to Academic Rules of the Institution. It also deals with multiple documents in multiple formats and also dynamically allocates questions across Courses Learning Outcomes. Most importantly, Teacher Assist supports the Core tenets of Outcome Based Education by carrying out a Gap Analysis of Individual student performance through analysis of Marks obtained by students through assessment results provided by Faculty. It then Identifies the areas where the student is deficient and identifies CLO wise deficiency based on predefined thresholds. It dynamically generates academic support for students through course material so students can meet the academic needs identified through the tool. Teacher Assist also converts course content into formats supportive to the institution's administration such as processing Faculty's uploaded content and transforming it to Institutional formats in structures such as a CSV or Template formats and more, all without modifying the uploaded material. Teacher Assist possesses extraordinary scalability utilizing modern Client Server Architecture. It has a web-based GUI built on Angular frameworks. Its Backend is built utilizing Fast API framework, with database integration and supports most used AI APIs throughout. It therefore can be hosted in universities and is designed to support the goals and objectives of Higher Education Commission as well as Outcome Based Education principles. Ultimately, it saves Faculty a lot of precious time and increases overall consistency and effectiveness of their Assessment practices while also ensuring higher quality and objectivity as well as empowering Data-Driven Decisions.

Keywords: Outcome-Based Education, CLO Mapping, Bloom's Taxonomy, NLP, Educational Technology.

Table of Contents

Declaration.....	i
Acknowledgments	ii
List of Tables	viii
List of Figures.....	ix
List of Abbreviations	x
List of Symbols.....	xi
CHAPTER 1	13
1. Introduction	13
1.1. CLO-Based Assessment	14
1.2. CIS Based Assessment Generation	14
1.3. CLO-Based Performance Tracking and Gap Analysis.....	14
1.4. CLO and GA Attainment Monitoring for Chairperson	15
1.5. KPI and Continuous Improvement	15
CHAPTER 2.....	16
2. Background and Literature Review	16
2.1. Literature Review Based on Research Papers	16
2.2. Literature Review Based on Feedback of Faculty	18
2.2.1. Section A: Features in Current OBE Portal.....	18
2.2.2. Section B: Proposed Features for a new OBE Portal System.....	18
2.2.3. Overall Feedback	19
2.3. Comparison of Existing Technology and Teacher Assist	23
CHAPTER 3	24
3. Aim and Statement of Problem	24
3.1. Problems and How We Tackled It.....	24
3.1.1. Manual Assessment Generation and CLO Tagging	24
3.1.2. Bloom and CLO Alignment Issues.....	25
3.1.3. Lack of Integrated Transcript Creation and Management.....	25
3.1.4. Difficulty in Gap Analysis and Weak Student Identification.....	25
3.1.5. No Proper Student Task Follow-Up System	26
3.1.6. Limited HOD Level Academic Monitoring	26
3.1.7. Limited Technological Familiarity Among Users.....	26
3.1.8. Data Storage, File Handling, and Retention	27
3.2. Key Changes in Project Aims.....	27
3.2.1. Role-Based Scope Expansion.....	27

3.2.2. Assessment Generation Upgrade.....	27
3.2.3. CLO and Bloom Mapping Improvement	27
3.2.4. PLO/PEO Focus Removed	27
3.2.5. Transform Module Added	27
3.2.6. Gap Analysis Strengthened	28
3.2.7. Student Task Management Added.....	28
3.2.8 HOD Insights Added	28
3.2.9. AI Assistant Added.....	28
3.2.10. Technology Stack Updated.....	28
3.2.11. Deferred or Removed	28
CHAPTER 4.....	29
4. Hardware, Software Analysis, And Requirements.....	29
4.1. Fact-Finding Techniques Used.....	29
4.1.1. Interviews:	29
4.1.2. Questionnaires:	29
4.1.3. Observation:	29
4.1.4. Document Analysis:	29
4.2. Software technologies.....	29
4.3. System Requirements in Standard User Story Format	30
4.3.1. User Authentication.....	30
4.3.2. Automated CLO–GAs Mapping.....	30
4.3.3. CLO Attainment Calculation.....	30
4.3.4. GAs Attainment Evaluation.....	31
4.3.5. KPI Analysis.....	31
4.3.6. Report Generation.....	31
4.3.7. Cloud Data Storage and Security.....	31
4.4. Diagrams.....	32
4.4.1. System Diagram	32
4.4.2. Activity Diagram:	33
4.4.3. Use case Diagram:	34
CHAPTER 5.....	35
5. Software Design and Modelling	35
5.1. Project Architecture:.....	35
5.2. UML Diagrams.....	38
5.2.1. Class Diagram	38

5.2.2. Sequence Diagram	41
5.2.3. Object Diagram:	44
5.3. Interface:	47
CHAPTER 6	57
6. Algorithm Analysis and Complexity	57
6.1. Algorithm	57
6.2. Pseudo Code	57
6.2.1. Pseudocode of Generation Module:	57
6.2.2. Pseudocode of Gap Analysis Module:	58
6.2.3. Pseudocode of Transformation Module:	60
6.3. Time Complexity	61
6.4. Retrieval-Augmented Generation	61
6.5. Text Embedding	61
6.6. Vector Similarity Search Cosine Similarity	62
6.7. Large Language Model Generation	62
CHAPTER 7	63
7. Implementation	63
7.1. System Overview	63
7.2. Code overview	63
7.2.1. Transformation Module Main.py	63
7.2.2. Generation Module App.py	64
7.2.3. Gap Analysis Module Main.py	66
7.3. Component Diagram	70
7.4. Deployment Diagram	70
CHAPTER 8	72
8. Testing	72
8.1. Introduction	72
8.2. Testing Plan	72
8.2.1. Testing Scope	72
8.2.2. Testing Types Used	72
8.2.3. Tools Used for Testing	72
8.3. Black Box Testing	73
8.3.1. Login Module Testing	73
8.3.2. File Upload	73
8.3.3. Weak Students + Generation + Transformation	73

8.4. White Box Testing.....	74
8.4.1. Threshold Calculation Logic	74
8.4.2. CLO Mapping Logic	75
8.4.3. Student Processing Loop	75
8.4.4. File Validation Logic.....	75
CHAPTER 9.....	77
9. Achievements	77
CHAPTER 10.....	78
10. Discussion.....	78
CHAPTER 11	82
11. Future Work.....	82
CHAPTER 12.....	86
12. Conclusion.....	86
CHAPTER 13.....	88
13. Appendices	88
13.1. Functional Requirements:.....	88
13.2. Non-Functional Requirements:.....	88
13.3. Hardware requirements:	88
13.4. Software requirements:.....	89
13.5. Input and Output Requirements.....	89
13.6. Supported File Formats	90
13.7. Module Description	90
13.8. Software Requirements.....	91
13.9. Bloom’s Taxonomy Levels Used	91
CHAPTER 14.....	92
14. References	92

List of Tables

Table 2.1: Comparison Between Technology	23
Table 4.1: Software Technologies used in Teacher Assist.....	29
Table 8.1: Login Module Test Cases.....	73
Table 8.2: File Upload Test Cases.....	73
Table 8.3: Weak Students and Generation and Transformation Test Cases	73
Table 8.4: Threshold Calculation Logic Test Cases.....	74
Table 8.5: Clo Mapping Logic Test Cases	75
Table 8.6: Student Processing Loop Test Cases.....	75
Table 8.7: Validation Logic Test Cases	76
Table 13.1: Functional requirements for Teacher Assist.....	88
Table 13.2: Non-Functional requirements for Teacher Assist	88
Table 13.3: Hardware requirements for Teacher Assist	88
Table 13.4: Software requirements for Teacher Assist	89
Table 13.5: Input and Output Requirement for Teacher Assist.....	89
Table 13.6: Supported File Format for Teacher Assist	90
Table 13.7: Module Description for Teacher Assist.....	90
Table 13.8: Software Requirements for Teacher Assist	91
Table 13.9: Blooms Taxonomy Levels Used in Teacher Assist.....	91

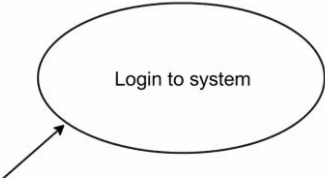
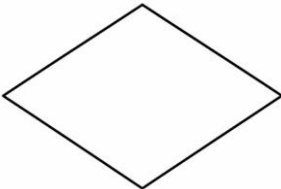
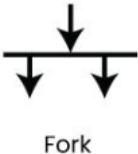
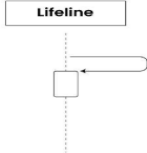
List of Figures

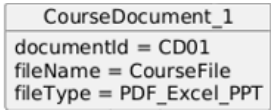
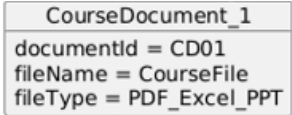
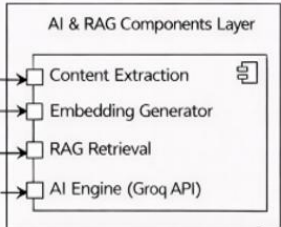
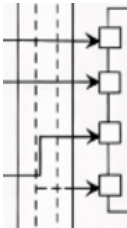
Figure 2.1: Importance of Bloom’s Taxonomy–Based Emphasis Detection	19
Figure 2.2: Usefulness of Automatic CLO/PLO Mapping.....	19
Figure 2.3: Benefits of CLO–PLO Accuracy Verification.....	20
Figure 2.4: System Effectiveness in Emphasis Level Classification.....	20
Figure 2.4: Satisfaction with CLO–PLO Interface	21
Figure 2.5: Ease of CLO Selection and Emphasis.....	21
Figure 2.6: Accuracy of Weightage Distribution	22
Figure 2.7: Overall Faculty Experience with the OBE Portal	22
Figure 4.1: System Diagram for Teacher Assist.....	32
Figure 4. 2 Activity Diagram for Teacher Assist	33
Figure 4. 3 Use Case Diagram for Teacher Assist	34
Figure 5.1: Generation Module of Teacher Assist	39
Figure 5.2: Gap Analysis Module of Teacher Assist	39
Figure 5.3: Transformation Module of Teacher Assist	40
Figure 5.4: Generation Module of Teacher Assist	41
Figure 5.5: Gap Analysis Module of Teacher Assist	42
Figure 5.6: Transformation Module of Teacher Assist	43
Figure 5.7: Generation Module of Teacher Assist	44
Figure 5.8: Gap Analysis of Teacher Assist.....	45
Figure 5.9: Transformation of Teacher Assist.....	46
Figure 5.10: Teacher Assist Login Interface	47
Figure 5.11: Profile Setup for Teacher Assist	47
Figure 5.12: Document Upload Interface for Exam Generation	48
Figure 5.13: Exam Configuration Interface.....	48
Figure 5.14: AI Prompt Generation Interface.....	49
Figure 5.15: Generated Exam Editing and Saving Interface	50
Figure 5.16: Transcript Generation Interface	50
Figure 5.17: Gap Analysis Upload Interface.....	50
Figure 5.18: Gap Analysis Result Interface	51
Figure 5.19: CLO Performance Chart	51
Figure 5.20: Question-Wise Gap Analysis Table.....	52
Figure 5.21: CLO-Wise Gap Analysis Summary	52
Figure 5.22: Students Requiring Attention Interface	52
Figure 5.23: Student Login Interface of Teacher Assist.....	53
Figure 5.24: Student Task Queue Interface	53
Figure 5.25: Chairperson Login Interface of Teacher Assist	54
Figure 5.26: Chairperson Course Assignment Interface	54
Figure 5.27: Admin Login Interface.....	55
Figure 5.28: Chairperson Account Creation Interface.....	55
Figure 5.29: Student Access Management Interface	56
Figure 5.30: Course Catalog Management Interface.....	56
Figure 7.1: Component Diagram for Teacher Assist.....	70
Figure 7.2: Deployment Diagram for Teacher Assist.....	71

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
CIS	Course Information Sheet
CLO	Course Learning Outcome
GAs	Graduate Attributes
GUI	Graphical User Interface
HEC	Higher Education Commission
NCEAC	National Computing Education Accreditation Council
NLP	Natural Language Processing
OBE	Outcome Based Education
RAG	Retrieval-Augmented Generation

List of Symbols

Symbols	Description
	The terminator symbol is used to indicate the beginning or the end of the system. Typically, it will include the term "Start" or "End."
	A decision or branching point. There are four points of the diamond where lines representing various decisions come out.
	The initial state is used to represent the state before an activity is conducted.
	To support concurrent activities, Fork nodes are used.
	A vertical dashed line representing the existence of an object over time.

	A variable or characteristic belonging to a class.
	A function or behavior the class performs.
	Aggregation: Hollow diamond → whole-part (weak relation).
	A hollow arrow pointing from child class to parent class.
	A rectangular box with two small tabs on the side showing a software module.
	A solid line showing network connection between nodes.

CHAPTER 1

1. Introduction

Teacher Assist is an academic support system that uses artificial intelligence to support the efficient generation of assessments, tracking student performance, transforming transcripts, and conducting gap analysis for university teachers. The system alleviates the academic burden on teachers by helping them create assessments from uploaded course materials, link questions with CLOs with the Course Information Sheet, store student tasks and analyze student performance with respect to learning outcomes.

The system assists the teachers to prepare the assessment papers with uploaded source material like PDF, DOC, DOCX, PPT, PPTX. The teacher posts the Course Information Sheet in addition to the source material. The system fetches the essential academic information from the CIS including Course Name, Course Code, CLO Descriptions, Bloom's taxonomy levels. This information is then used to map generated questions with the appropriate CLOs and cognitive levels. The system also alerts teachers if the selected Bloom levels are greater than the mapped CLO level.

Teacher Assist has a Generate Module for teachers to make quizzes, midterm exams, final exams and assignments. The created paper contains question number, marks, Bloom's level, and CLO tagging. The system relies on AI with Llama 3.3 70B for creating exams via text. It also offers local image analysis with Ollama/Lava when documents contain images but primarily relies on cloud-based AI generation.

Teachers can develop and manage transcripts using the Transform Module. Teachers can choose any registered course, enter a mark and create an excel transcript. These are created transcripts which are stored in the system for 28 days and teachers can download, update or delete these transcripts during this period. The saved transcript records and files get automatically removed after 28 days. The module also allows for updating of transcripts when student task reviews are marked as pass or fail, matching records with the student roll number.

Gap Analysis Module is used by the teacher to analyze the student performance by uploading question papers and transcripts. It detects weak students, CLO-wise gaps and student performance patterns. Visual analytics like charts and a question-wise student performance heatmap enable teachers to easily spot weak areas and performance trends.

Teacher Assist also has a Student Task Management feature. Students can receive practice problems, reference files, and for the teachers to check their work. Students can see their tasks, download files attached to them, submit answers, upload files attached to their answers, and change their password. Student task records, notifications and attachments submitted are also retained for 28 days and then automatically deleted.

There is role based access for Admin, Chairperson, Teacher, and Student user. Admins and Chairperson can add, edit, and delete users, courses, teachers, and student records. Teachers can edit

their profile, courses they are registered for, generate and transform assessments, view gap analysis, and review student tasks. Students have access to assigned tasks and can manage their password.

Teacher Assist offers an AI assistant within the app to assist users with questions related to the system, considering the context of the page. The assistant is based on Llama 3.3 70B and is designed to answer based on the visible module or page the user is working on.

In general, Teacher Assist is an academic support system that is practical and uses AI to create assessments, map questions with CLOs, create transcripts, analyze learning gaps, assign student tasks, and monitor student performance. The system aims to minimize academic work in manual mode, standardize assessment, facilitate CLO assessment, and assist teachers in making academic decisions based on data.

1.1. CLO-Based Assessment

This module will enable teachers to use the uploaded Course Information Sheet and course material to facilitate the efficient management of CLO-based assessment. The system pulls out CLOs, CLO descriptions, course name, course code, and Bloom's taxonomy levels from the CIS. These extracted CLOs are then used to map generated assessment questions with the correct CLO and cognitive level.

Teachers can create quizzes, assignments, midterm exams and final exams based on uploaded course content. Each question or section on the generated paper has CLO tagging, marks and Bloom's level. The system also alerts the teacher if the selected Bloom is higher than the mapped CLO.

1.2. CIS Based Assessment Generation

Teacher Assist reads the Course Information Sheet and course material to get to know the structure of the course, CLOs, weekly topics and Bloom's cognitive levels. The system can retrieve CLOs, Course information, and weekly coverage from the CIS and utilize this information for assessment generation and CLO mapping.

Teachers can create quizzes, assignments, mid-term exams and final exams based on uploaded course content. The assessments generated contain the related CLOs, Bloom's levels, marks and question structure based on the selected assessment type. The system also determines if the chosen Bloom level is appropriate for the mapped CLO, and alerts for warning if necessary.

1.3. CLO-Based Performance Tracking and Gap Analysis

Teacher Assist enables teachers to track student achievement against CLOs using transcript transformation and gap analysis. From uploaded course content, teachers can create quizzes, assignments, midterm exams, and final term exams, each of which is tagged to the appropriate CLO and cognitive level of Bloom's taxonomy.

Once assessments are done, teachers can upload or create assessment results and utilize the Gap Analysis module to check student performance. The system identifies weak students, analyzes the performance on the basis of CLO and generate visual reports like charts and Question-wise student

performance heat map. These reports enable teachers to be aware of students who are having difficulties and which areas of the CLO require more attention.

1.4. CLO and GA Attainment Monitoring for Chairperson

Teacher Assist enables outcome-based academic management by enabling teachers and Chairperson to track the attainment of CLO and GA via assessment data and performance analysis. Teachers can create assessments based on uploaded course content, link questions to CLOs, and gain insights from student performance via transcripts and gap analysis reports.

The system also gives Chairperson level insights which include teacher-wise performance, course-wise CLO attainment and GA attainment. The system maps classroom assessments to graduate attributes, or CLOs, providing HODs with a way to determine students' progress toward desired academic outcomes.

Teacher Assist provides valuable information to support academic decision making, including identification of weak CLOs, low-performing students, and areas requiring further teaching support. These insights enable continuous improvement as teachers and Chairperson have clear data on the performance of courses and achievement of learning outcomes.

1.5. KPI and Continuous Improvement

Teacher Assist helps teachers and HODs to identify learning gaps from the assessment results, CLO attainment and insights on GA attainment for continuous academic improvement. The system will analyze the transcript of students, question-wise performance and will identify the weak students, weak CLO, and need for further academic support.

Gap analysis reports, visual charts and the question-wise student performance heatmap help teachers to understand what needs improvement for the topics or CLOs. Chairperson level insights also enable departments to see the teacher-wise, course-wise, CLO-wise and GA-wise attainment trends.

The system assists in the continuous improvement cycle by offering evidence based on student performance data, task submissions, transcript records and assessment outcome. Weak students can be given follow-up practice tasks and their answers can be checked by teachers to close the gaps in learning identified.

CHAPTER 2

2. Background and Literature Review

Outcome Based Education is a concept that has been gaining importance in Higher Education, as it focuses on assessing students' skills acquired at the end of the course. The Course Learning Outcomes, Bloom's taxonomy levels and Graduate Attributes are used in an OBE academic environment to synchronize teaching and assessment, as well as performance. This alignment is used for institutions to track if students are reaching the expected learning outcomes and if courses are meeting the program-level goals.

It has been revealed from the existing literature that many universities have adopted the following assessment, calculation and reporting systems to support the OBE implementation in their institutions: use of CLO-based assessment methods, use of CLO attainment calculation and, use of manual or semi-automated reporting systems. These will assist teachers and departments to assess students' learning and determine areas of underachievement. However, many of these systems rely on spreadsheets, separate documents and manual mapping of questions with CLOs which can lead to increased teacher workload and decreased accuracy.

The current advancements in Artificial Intelligence and Natural Language Processing also have opened up new opportunities for academic automation. The use of AI-driven systems can aid in creating evaluations based on course material, identifying valuable content in documents, and assisting educators with time-consuming academic tasks. But most of the current solutions only cater to a single component of the Academic process, like creation of assessments or calculation of attainment of CLOs, management of transcript or performance analysis.

Hence, it is required to have an integrated academic support system with AI based assessment generation, CLO/ Bloom mapping, transformation of transcript, gap analysis, student task review and HOD based attainment monitoring system. To fill this gap, Teacher Assist is built to be a single platform for teachers, Chairperson, admins and students, and supports CLO to GA mapping and academic performance monitoring.

2.1. Literature Review Based on Research Papers

The rationale behind the use of Outcome Based Education in higher education is that it measures learning outcomes rather than just course completion. The Course Learning Outcomes, and the Bloom's taxonomy levels and Graduate Attributes are used to help align teaching, assessment, and monitoring of academic performance in OBE. Yet, in many institutions, some of the activities related to OBE – CLO mapping, preparation of assessment, marking and management of transcript, gap analysis, and reporting of attainment report – are still being done manually or by using separate tools. This leads to an increase in teacher's workload and can result in mapping errors, performance calculations and academic reports errors.

Arafah [1] suggested a curriculum mapping tool for higher education which can be used to study the structure of the course content covering topics taught, teaching sequence and depth of coverage. This study will be helpful for the curriculum improvement of aligning and cohering curriculum in that it enables the teacher to visualize how the curriculum is organized. However, the tool primarily addresses curriculum structure and is not oriented towards measures of student performance, CLO attainment and identification of learning gaps. TeacherAssist takes this one step further by incorporating course

details, learning outcomes, and linking them to AI driven assessment generation, CLO tagging, mark sheet handling, gap analysis and HOD level academic monitoring.

Chowdhury et al. [2] created an excel based assessment and evaluation tool for a bachelor's level program using an OBE approach. They use direct assessment met chairperson, like quizzes, homework or tests, to determine CLO and program outcome attainment. This work shows the importance of using assessment results for outcome evaluation and continual quality improvement. Nevertheless, the system is far from data-driven and is not very automated, scalable or academic-monitored in real-time. Teacher Assist, on the other hand, is an academic support system developed as a web-based solution, which automates numerous teacher processes such as the generation of assessment, creating transcripts, gap analysis, student task review, and the generation of attainment insights.

In an undergraduate computing program, Shaheen [3] suggested analyzing the relationship between CLOs and program learning outcomes with a decision tree approach. To identify influential CLOs and to enhance the accuracy of the outcome analysis, statistical analysis and C4.5 decision tree algorithm were applied. According to this research, the use of data-driven approaches is important in implementing OBE. The approach, however, is primarily based on structured numerical data and is specific to a computing course context. Teacher Assist offers a more integrated approach by incorporating extracted CIS data, Bloom's taxonomy levels, CLO descriptions, question performance and teacher transcripts to assist with analysis and GA-level academic insights based on the CLO.

To deal with manual mapping of the CLO to the outcomes, Zaki et al. [4] suggested a framework based on NLP which uses Cosine similarity and thresholding to compare the text of learning outcomes. Their research demonstrated that NLP can decrease human error and enhance the precision of educational outcome mapping. This study is also like Teacher Assist, as it also employs logic to match generated questions with the CLO descriptions, Bloom levels, question types, and topic relevance through the use of AI. Teacher Assist, however, does not just perform text similarity, it uses this mapping process to generate assessments, generate warnings for mismatch between Bloom and CLO, transform transcripts, generate gap analysis reports and monitor CLO to GA at HOD level.

Bloom et al. [5] have developed Bloom's Taxonomy of educational objectives, that categorizes educational objectives at 6 levels of cognitive level. Bloom's taxonomy is still relevant in the design of assessment as it enables teachers to plan questions to various thinking levels including remembering, understanding, applying, analyzing, evaluating, and creating. Bloom's taxonomy is used by Teacher Assist in designing assessments so that questions are tailored to the appropriate cognitive level. The system also alerts teachers if the selected Bloom level is higher than the mapped CLO level, thus helping to keep the assessment aligned with OBE.

Recently, Retrieval-Augmented Generation [6] was introduced to enhance the language generation task by first retrieving information that is relevant to the task before generating the answer. This is helpful for education systems, in that generated content should be tied to the course content provided and not create unrelated content. Teacher Assist works in a manner like Knowledge Grounded, in that it creates quizzes, assignments, midterm and final exams from uploaded course files and CIS data. This facilitates the creation of assessments that reflect content of the course and the appropriate learning outcomes.

Alahmari et al. [7] talked about the problems of implementing the OBE in Higher Education. OBE implementation comes with challenges such as the manual workload, the need for manualization, the complexity of outcome mapping and reporting, as highlighted in their study. These difficulties argue for the need for systems that can lessen manual academic effort and increase accuracy in outcome-based monitoring. Teacher Assist tackles these challenges by offering AI-driven assessment generation, CLO and Bloom mapping, Excel transcript management, gap analysis, student task management and CLO and GA attainment monitoring at HOD-level in one platform.

In summary, the literature reviewed indicates that the current research has significantly contributed to curriculum mapping, calculation of OBE assessment, outcome mapping, NLP-based CLO mapping, designing assessment for learning, and generating content using AI. Most current systems however are designed for one aspect of the academic process - curriculum mapping, spreadsheet-based attainment calculation, or CLO-to-outcome mapping. There is a need for an integrated academic support system that will connect the assessment generation, CLO and Bloom alignment, transcript transformation, weak student task assignment and academic monitoring at HOD level. This gap is addressed by Teacher Assist's role-based, AI-driven platform which supports students, teachers, Chairperson, and Admins, with support for CLO attainment, mapping of CLO to GA, and continuous academic improvement.

2.2. Literature Review Based on Feedback of Faculty

2.2.1. Section A: Features in Current OBE Portal

Q1. How satisfied are you with the CLO-PLO attachment process?

Q2. How easy is it to specify CLOs and map them to PLOs?

Q3. How effective is the system in defining emphasis levels (Cognitive / Psychomotor / Affective)?

Q4. Are you satisfied with the clarity and design of the CLO-PLO interface?

Q5. How easy is it to select and emphasize specific CLOs within the system?

Q6. How accurate is the weightage (100%) distribution in the current system?

2.2.2. Section B: Proposed Features for a new OBE Portal System

Q1. How useful would it be if the system allowed CIS uploads (so that CLO/PLO mappings are updated automatically)?

Q2. How beneficial would a CLO-PLO Accuracy Verification system (e.g. sync with HEC data or validation checks) be?

Q3. How important do you think it is to have automatic emphasis level detection based on Bloom's Taxonomy keywords?

2.2.3. Overall Feedback

Q1. How would you rate your overall experience with the OBE portal so far?

Q2. How likely are you to recommend this system to other faculty members?

Q3. What improvements or additional features would you like to see in the OBE portal?

How important do you think it is to have automatic emphasis level detection (based on Bloom's Taxonomy keywords)?

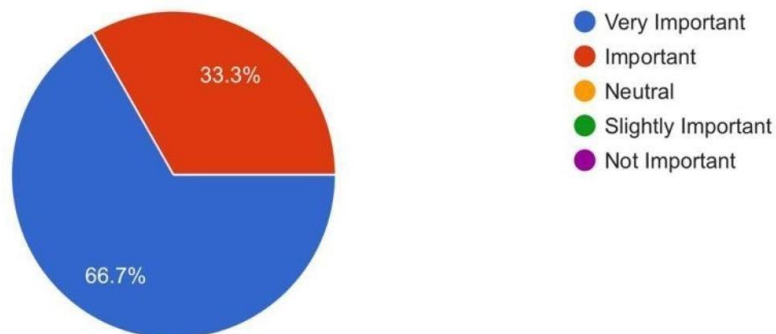


Figure 2.1: Importance of Bloom's Taxonomy-Based Emphasis Detection

- Most faculty (66.7%) consider automatic Bloom's level detection very important, while 33.3% consider it important.

How useful would it be if the system allowed CIS Uploads (so that CLO/PLO mappings are updated automatically)?

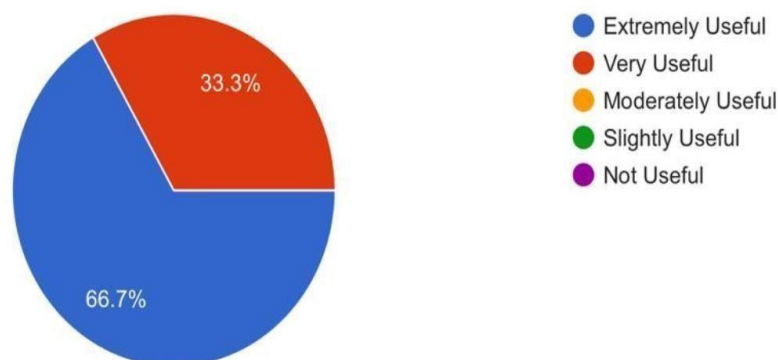


Figure 2.2: Usefulness of Automatic CLO/PLO Mapping

- Majority (66.7%) find CIS uploads extremely useful, while 33.3% consider them especially useful.

How beneficial would a CLO–PLO Accuracy Verification system (e.g., sync with HEC data or validation checks) be?

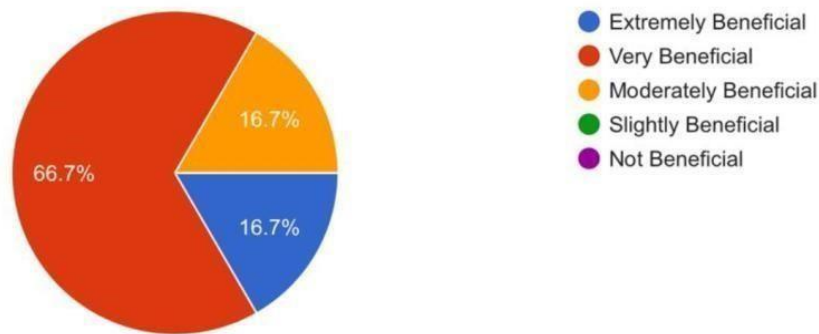


Figure 2.3: Benefits of CLO–PLO Accuracy Verification

- Most faculty (66.7%) consider the CLO–PLO accuracy verification system very beneficial, while 16.7% rate it extremely beneficial and 16.7% moderately beneficial.

How effective is the system in defining emphasis levels (Cognitive / Psychomotor / Affective)?

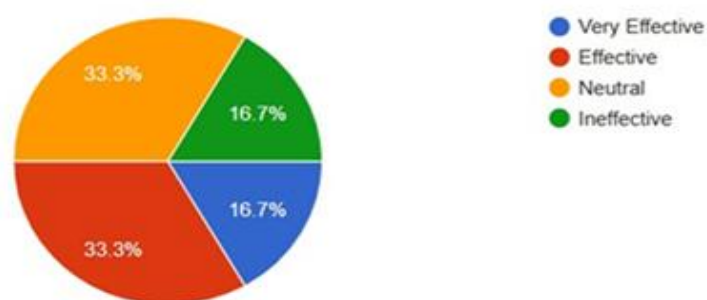


Figure 2.4: System Effectiveness in Emphasis Level Classification

- Most faculty respondents viewed the system as effective in defining emphasis levels, with 33.3% rating it very effective, 33.3% effective, 16.7% neutral, and 16.7% ineffective.

Are you satisfied with the clarity and design of the CLO–PLO interface?

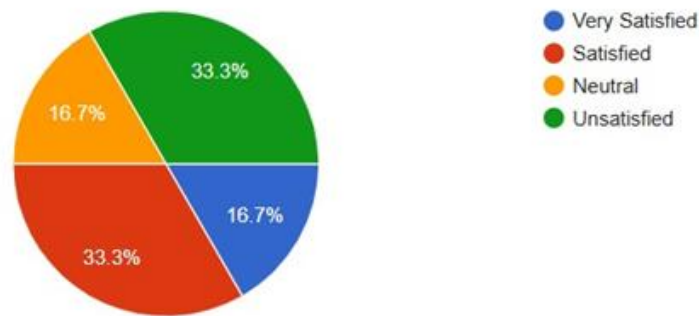


Figure 2.4: Satisfaction with CLO–PLO Interface

- Most faculty respondents were satisfied with the system overall, with 16.7% very satisfied, 33.3% satisfied, 16.7% neutral, and 33.3% unsatisfied.

How easy is it to select and emphasize specific CLOs within the system?

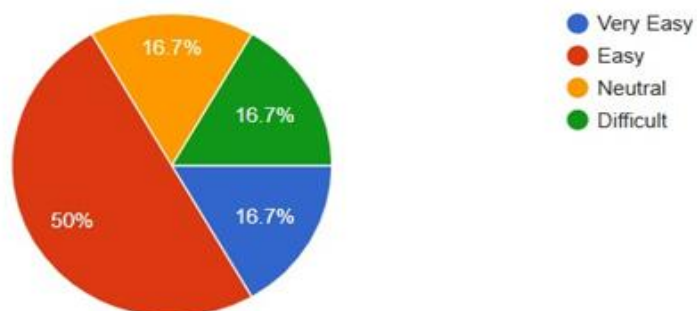


Figure 2.5: Ease of CLO Selection and Emphasis

- Most faculty respondents found the system easy to use for selecting and emphasizing specific CLOs, with 16.7% rating it very easy, 50% easy, 16.7% neutral, and 16.7% difficult.

How accurate is the weightage (100%) distribution in the current system?



Figure 2.6: Accuracy of Weightage Distribution

- Faculty responses indicated that the weightage (100%) distribution in the current system was generally accurate, with 50% rating it mostly accurate and 50% sometimes accurate

How would you rate your overall experience with the OBE portal so far?

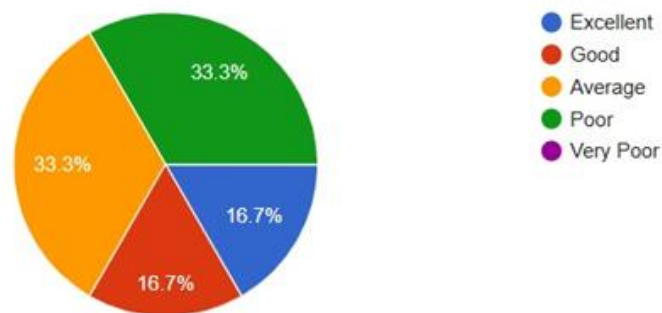


Figure 2.7: Overall Faculty Experience with the OBE Portal

- Faculty feedback on the OBE portal experience was mixed, with 16.7% rating it excellent, 16.7% good, 33.3% average, and 33.3% poor

2.3. Comparison of Existing Technology and Teacher Assist

[Table 2.1](#) compares existing OBE systems. It shows that earlier systems offer limited features, while Teacher Assist provides a more complete and integrated solution.

Table 2.1: Comparison Between Technology

System / Source	Curriculum Alignment	Automated CLO/PLO	CLO–PLO Detection	PLO Prediction	OBE Automation
Curriculum Mapping Tool [1]	yes	no	no	no	no
Excel-Based OBE Assessment System [2]	no	yes	no	no	no
Decision Tree Model for CLO–PLO Mapping [3]	no	no	yes	no	no
OBE Prediction Framework [4]	no	no	no	yes	no
Teacher Assist (Proposed System)	yes	yes	yes	no	yes

CHAPTER 3

3. Aim and Statement of Problem

The main goal of the Teacher Assist project is to build a web application based on Artificial Intelligence for academic support and assessment management in the context of Outcome Based Education in HEIs. The system aims at minimizing teacher man-hours in generating assessment, accuracy in producing assessment based on CLO, ease in the process of handling transcript, identification of student learning gaps, design and assign of improvement tasks to the student and access to CLO and GA attainment insights for chairperson level academic monitoring.

Many activities are still being done manually, which may pose challenges for university teachers in handling academic tasks related to OBE. These tasks involve preparing assessments, mapping questions with CLOs, checking the alignment against Bloom's taxonomy, maintaining transcripts, analyzing student performance, identifying weak students, and preparing academic performance summaries. These processes must be performed manually which can lead to errors in calculation, reporting and evaluating learning outcomes, adds workload and causes delay.

Current school education systems primarily emphasize content dissemination, attendance, or simple grading. They lack integrated workflow to generate assessments based on AI, extract CLOs from CIS, check for Bloom/CLO mismatch, transform mark sheets, do gap analysis, review student tasks, and monitor CLOs and GA at chairperson level. Hence, there is a need for a system that can support teachers, students, chairperson and admins with a central AI based platform for academic automation and performance analysis.

3.1. Problems and How We Tackled It

The aim of this section is to describe the major issues that were tackled in the development of Teacher Assist and how these are met by the system. The problems depicted represent practical problems of academic challenges in OBE-based teaching and assessment management.

3.1.1. Manual Assessment Generation and CLO Tagging

3.1.1.1. Problem:

Typically, teachers invest a considerable amount of time to create quizzes, assignments, midterm exams and end-of-term exams manually. Another function they are required to perform is to make sure that every question is associated with the right CLO and Bloom's taxonomy level. This can take a lot of time to do manually, and may lead to inaccurate CLO tagging, lack of topic coverage, or the wrong difficulty for a given CLO.

3.1.1.2. How We Tackled It:

Teacher Assist includes an AI-powered Generate module that teachers upload course material and Course Information Sheet. The system pulls out relevant CIS information including course code, course name, CLO numbers, CLO descriptions, Bloom levels and weekly topics (if available). The system creates quizzes, assignments, midterm and final exams with marks, question numbers, Bloom taxonomies and CLO tags based on the uploaded material and CIS. The system also alerts teachers if there is a mismatch in Bloom level selected and mapped CLO level.

3.1.2. Bloom and CLO Alignment Issues

3.1.2.1. Problem:

When using an OBE approach to assessment, questions need to be mapped to the appropriate CLO and Bloom's taxonomy level. The teachers can sometimes choose a Bloom level which is not exactly that of the CLO level, making the assessment design inconsistent and impacting the quality of the learning outcome evaluation.

3.1.2.2. How We Tackled It:

With the help of CIS-based CLO descriptions, Bloom levels, question type and topic relevance, Teacher Assist matches generated questions with appropriate CLOs. During assessment generation, the system verifies the Bloom/CLO alignment and provides a visible warning if the selected Bloom level does not correspond to the mapped CLO level. This will allow teachers to read through the evaluation and correct it before saving or downloading it.

3.1.3. Lack of Integrated Transcript Creation and Management

3.1.3.1. Problem:

Transcripts are usually made and kept by teachers in excel. In cases where teachers require to update or reuse transcripts later, managing student transcripts, assessment details, course information, and saved files can be difficult.

3.1.3.2. How We Tackled It:

Teacher Assist has a Transform module where teachers can generate, save, update, download, and delete Excel transcripts. Teachers will be able to choose from a list of registered courses, add assessment information, and enter marks for students. Transcripts saved will be retained for 28 days. Teachers can open, download, update, and delete them during this time. Records and supporting files of saved transcript are automatically removed after 28 days.

3.1.4. Difficulty in Gap Analysis and Weak Student Identification

3.1.4.1. Problem:

Manual analysis of student performance is time consuming and challenging. Teacher to monitor marks (question wise), achievement (CLO wise), weak students, learning gap etc. It is not easy to determine who needs assistance and which CLOs need improvement without an appropriate analysis.

3.1.4.2. How We Tackled It:

Teacher Assist offers a Gap Analysis module that allows teachers to upload or use a question paper, transcript and CIS as required. The system gives analysis of students mark, CLO-wise analysis, weak students, learning gaps, and question-wise analysis. It displays results in the format of charts, CLO summaries, weak student lists and a question-wise student performance heatmap. The heatmap enables teachers to identify pertinent learning gaps with the CLO more clearly, as well as weak questions and weak students.

3.1.5. No Proper Student Task Follow-Up System

3.1.5.1. Problem:

Once weak students have been identified, teachers do not have a suitable system in place to assign improvement tasks, collect student submissions, review answers, and/or track student improvement. This undermines corrective action process in OBE.

3.1.5.2. How We Tackled It:

Student Task Management is a part of the Teacher Assist. Teachers can give tasks to low-ability students and attach reference files. Students can see who has been assigned to a task, download files, submit typed answers, upload attachments and view review status. The teacher will be able to see the submission and mark it PASS or FAIL, add a mark or comments, and then update the relevant transcript if the student's roll number is the same. Student task records, notifications, teacher attachments and student submission files are stored for 28 days and then automatically deleted.

3.1.6. Limited HOD Level Academic Monitoring

3.1.6.1. Problem:

The chairperson must track the academic achievement of teachers, courses, CLOs, GA, and departments. Where data is stored in each separate file or system though, it becomes difficult to identify weak courses, weak CLOs, low attainment areas and teachers or courses that require support.

3.1.6.2. How We Tackled It:

Teacher Assist offers chairperson level insights and attainment monitoring. The chairperson can see teacher-wise academic insights, course-wise performance, CLO attainment, GA attainment, and the trends of teacher performance for the department. This is facilitated by the mapping chain which consists of the following: question performance to CLO attainment and CLO attainment to GA attainment. The insights assist the chairperson better to make academic decisions and pinpoint areas in need of improvement.

3.1.7. Limited Technological Familiarity Among Users

3.1.7.1. Problem:

Teachers and academic users might not feel at ease with the use of more sophisticated digital and AI-based tools. These interfaces may be confusing and cause users to rely on manual documentation.

3.1.7.2. How We Tackled It:

The user interface of Teacher Assist is designed in a simple and role based manner for Admin, chairperson, Teacher and Student. Only those features and attributes are visible to each user that are relevant to his function. There's also a page-aware AI Assistant that can grasp what's on the page and what's visible on the records. It assists users when responding to questions about the current dashboard, system modules, and academic data available.

3.1.8. Data Storage, File Handling, and Retention

3.1.8.1. Problem:

When it comes to keeping up with academic records, generated assessments, transcripts, student work files and attached documents across multiple devices or spreadsheets can be challenging. Unnecessary storage over long-term can also allow for file management problems.

3.1.8.2. How We Tackled It:

Teacher Assist will be using Postgres as their production data store using Supabase. Uploaded files, generated PDFs, transcripts, task attachments are stored in the backend and record, and file references are stored in the database. An important retention rule is applied to Transform transcripts and student task records/files, which is 28 days. These records and other files are automatically deleted after 28 days.

3.2. Key Changes in Project Aims

3.2.1. Role-Based Scope Expansion

The first one was primarily geared towards teacher assessment generation and student performance analysis. The new version introduces four new roles: Admin, chairperson, Teacher and Student. Admin has control over chairperson, students and courses. Chairperson is responsible for teachers, assignments and academic insight. The main academic modules are accessed by teachers and students view and submit assigned tasks.

3.2.2. Assessment Generation Upgrade

Until now, the system was primarily designed to create quizzes and assignments. The new version now enables quizzes, assignments, midterm exams and final exams based on uploaded course materials and CIS data. The following information is provided for generated assessments: Course code, Course Name, Exam Type, Total Mark, Time Allowed, Marks per Question, Bloom's Level, and CLO Tags.

3.2.3. CLO and Bloom Mapping Improvement

The earlier version only included basic CLO tagging. The new system gets CLOs and Bloom levels from the CIS, maps questions based on their CLO description and Bloom's taxonomy level, question type, and topic relevance. The system also alerts teachers to the Bloom/CLO mismatch.

3.2.4. PLO/PEO Focus Removed

The mapping of CLO–PLO–PEOs and automated reporting, in the style of accreditation, were part of the previous project aim. The system has been updated such that the emphasis has been removed from PLO and PEO management. The current system is primarily geared towards the generation of assessment items, mapping of CLO to GA, assessment of GA and academic insights at the chairperson level.

3.2.5. Transform Module Added

The new system features a comprehensive Transform module to help you to develop and edit Excel transcripts. Teachers can create, save, update, download and delete transcripts. These are the

transcripts which are preserved for 28 days from the date of saving and will be automatically removed after this period.

3.2.6. Gap Analysis Strengthened

Student marks, CLO-wise performance, weak students, learning gaps, and question-wise performance have been improved in the Gap Analysis module. New visual outputs include a chart, CLO summary, weak student identification and a question-wise student performance heat map.

3.2.7. Student Task Management Added

Student task assignments and review are included in the updated system. Teachers can give work to weak pupils, pupils can hand in answers and attachments, teachers can look at and mark them as PASS or FAIL. Reviewed outcomes can be used to update transcripts that have been saved.

3.2.8. HOD Insights Added

An improved system to enable chairperson monitoring is now in place. Chairperson have a view of teacher-wise insights, course-wise performance, CLO attainment, GA attainment and department-level trends. This enables better academic supervision and helps to identify weak courses, weak CLOs and low attainment areas.

3.2.9. AI Assistant Added

The new system features an AI Assistant based on page-aware technology and Groq LLaMA 3.3 70B. The assistant knows what the page or module is currently displaying and can help based on what is visible on the page and on what functions are known to function in the system.

3.2.10. Technology Stack Updated

The final system is built with the frontend using Angular, HTML, CSS and TypeScript. Backend written with Python, Fast API and SQLAlchemy. Supabase is a production database that runs PostgreSQL. The front end is deployed on Vercel and the back end is deployed on Render Web Service. The AI used is Groq API (LLaMA 3.3 70B).

3.2.11. Deferred or Removed

The system does not currently have a module to deal with the management of PEO, a module to access alumni feedback, a module to access industry feedback, a module to automatically generate an NCEAC submission, a module to summarize lectures, and a module to run a separate student query assistant. These features can be considered in the future if it is necessary.

CHAPTER 4

4. Hardware, Software Analysis, And Requirements

4.1. Fact-Finding Techniques Used

To determine the needs of users and system requirements, several Fact-Finding techniques were employed.

4.1.1. Interviews: Interviewing instructors and program coordinators to determine challenges with CLO-GAs mapping, calculating KPIs, and generating reports.

4.1.2. Questionnaires: Sent to gather the input on the system usability and features that are desired (automation, report generation, accuracy).

4.1.3. Observation: The current manual process of inputting course data and computing OBE attainments was observed to be used to identify inefficiencies.

4.1.4. Document Analysis: Previous OBE reports, course outlines and CLO-GAs mapping sheets were analyzed to identify data fields.

4.2. Software technologies

[Table 4.1](#) is a selection of the technologies that were used in the development of Teach Assist. It outlines the end decisions of each layer as well as the rationale behind the choice based on performance, scalability and integration. In general, the chosen stack guarantees the reliable, efficient, and real-time OBE automation system.

Table 4.1: Software Technologies used in Teacher Assist

Category	Technology	Final choice	Justification
Front-End	- HTML - CSS - JavaScript - React.js - Angular	Angular	Angular was selected for its modular structure. It provides a scalable and responsive interface for users and integrates efficiently with Python and PostgreSQL.
Back-End	- Node.js - PHP - Python (Django / Flask) - ASP.NET	Python/Fast API	Provides a high-performance, nonblocking server environment that easily integrates with Angular and PostgreSQL.
Database	- Firebase - MySQL - PostgreSQL	PostgreSQL	Selected for real-time data synchronization, and server less deployment.

Simulation / Analytics Layer	-Node.js Logic - Python (NumPy / Pandas) - MATLAB / Simulink	- Python backend logic + Angular charts	Implemented for backend analytical operations such as CLO–PLO attainment and KPI computation. Ensures integrated, efficient, and real time performance within the same environment.
Algorithms	- Rule-Based Algorithm - Relational Mapping Algorithm - Natural Language Processing	- Rule-Based Algorithm - Relational Mapping Algorithm - Natural Language Processing	Used for Keyword extraction, text classification, transparent and verifiable attainment calculation and CLO–PLO linkage. Machine learning models are reserved for future predictive analysis enhancements.
Data Structures	- Arrays / Lists - Objects / Dictionaries - Firestore Collections - Relational Tables	- Arrays / Lists - Objects/ Dictionaries - Relational Tables	Arrays and objects handle temporary runtime computations, while Fire store collections provide permanent, scalable, and synchronized cloud data storage.

4.3. System Requirements in Standard User Story Format

4.3.1. User Authentication

As an instructor or Administrator, I want to log in securely using my credentials so that only authorized users can access the system’s features.

4.3.1.1. Acceptance Criteria:

Since I have registered account,

I can enter valid credentials and press the login button,

Then the system should authenticate the user and display the appropriate dashboard based on the assigned role.

4.3.2. Automated CLO–GAs Mapping

As an Instructor, I would prefer that each CLO should automatically be mapped with the related PLOs so that the process could be quick, accurate and consistent without manual input.

4.3.2.1. Acceptance Criteria:

Since there is course and program data in the database,

When I select a specific course, Then the system must automatically create or propose CLO Gas mappings based on pre-established relational rules or templates.

4.3.3. CLO Attainment Calculation

I would like the system to automatically calculate CLO attainment based on the uploaded student marks to enable me to efficiently assess student performance.

4.3.3.1. Acceptance Criteria:

Since the data on assessment and marks have been uploaded,

When I press on the button to calculate CLO Attainment, Then the system must utilize the rule-based algorithm and show percentages of attainment of CLOs along with graphical representation.

4.3.4. GAs Attainment Evaluation

As a teacher, I would like the system to compute the overall GAs achievement based on CLO results such that program-level achievements can be assessed to be accredited.

4.3.4.1. Acceptance Criteria:

Since there is CLO attainment and CLO-GAs mapping data available,

When I generate a report, Then the system should combine CLO data to calculate its attainment and present it in a visual format in charts and tables.

4.3.5. KPI Analysis

As an instructor, I want to see KPI analysis in such a way that I can understand the performance of the institution and the achievement of the academic targets.

4.3.5.1. Acceptance Criteria:

Since the system has already taken the results of CLO and GAs attainment,

When I open KPI analysis module, Then the system must provide the computed KPIs and present the visual trends with the help of dashboards and graphs.

4.3.6. Report Generation

As an Instructor, I would like the system to automatically create OBE reports to enable me to easily share performance summaries during evaluations or accreditation reviews.

4.3.6.1. Acceptance Criteria:

Since all CLO, GAs and KPI data are accessible,

As I press the button of generating report, Then the system must generate a structured report (PDF or Excel) consisting of institution name, course details, attainment results, and visual analytics.

4.3.7. Cloud Data Storage and Security

As an Administrator, I would like all the institutional and student data to be securely stored in the cloud allowing records to be safe, backed up and accessible at any time.

4.3.7.1. Acceptance Criteria:

As the system is based on a PostgreSQL DBMS, all the data entered and updated will be stored in structured relational tables, using Supabase to host the DBMS. It uses Fast API and SQL Alchemy for the backend, which ensures that all records, including users, courses, teacher profiles, student tasks, transcripts, and assessment data, are stored securely.

The system also has user-based access to different roles such as Admin, HOD, Teacher, and Student. Each user role has access to only the functions and data that are relevant to his or her role, which helps ensure that sensitive academic and student information is protected.

Database records contain references to uploaded files, generated transcripts, task attachments and assessment related files are stored in the backend file system. This helps to manage data in an organized manner and to control access to academic records.

4.4. Diagrams

4.4.1. System Diagram

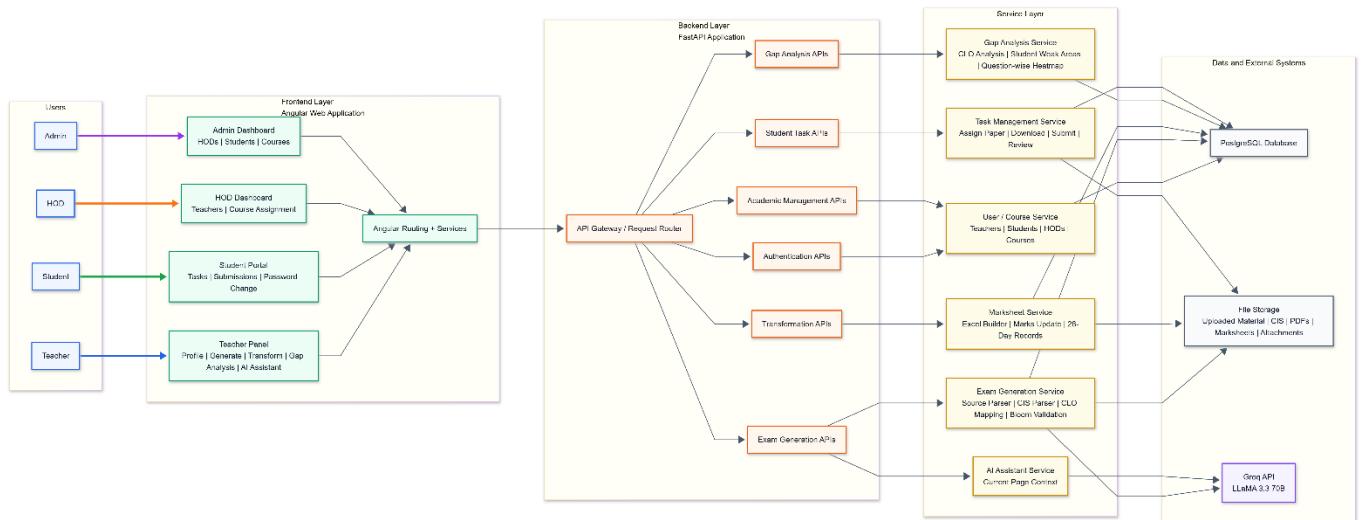


Figure 4.1: System Diagram for Teacher Assist

Figure 4.1 Firebase to authenticate, store files and manage database; and the Server Layer to extract and map CLO using trained NLP models. The process works in such a way that teachers upload syllabi and reports, admins review them, and automated analytics dashboards and KPI reports are generated after processing and validation.

4.4.2. Activity Diagram:

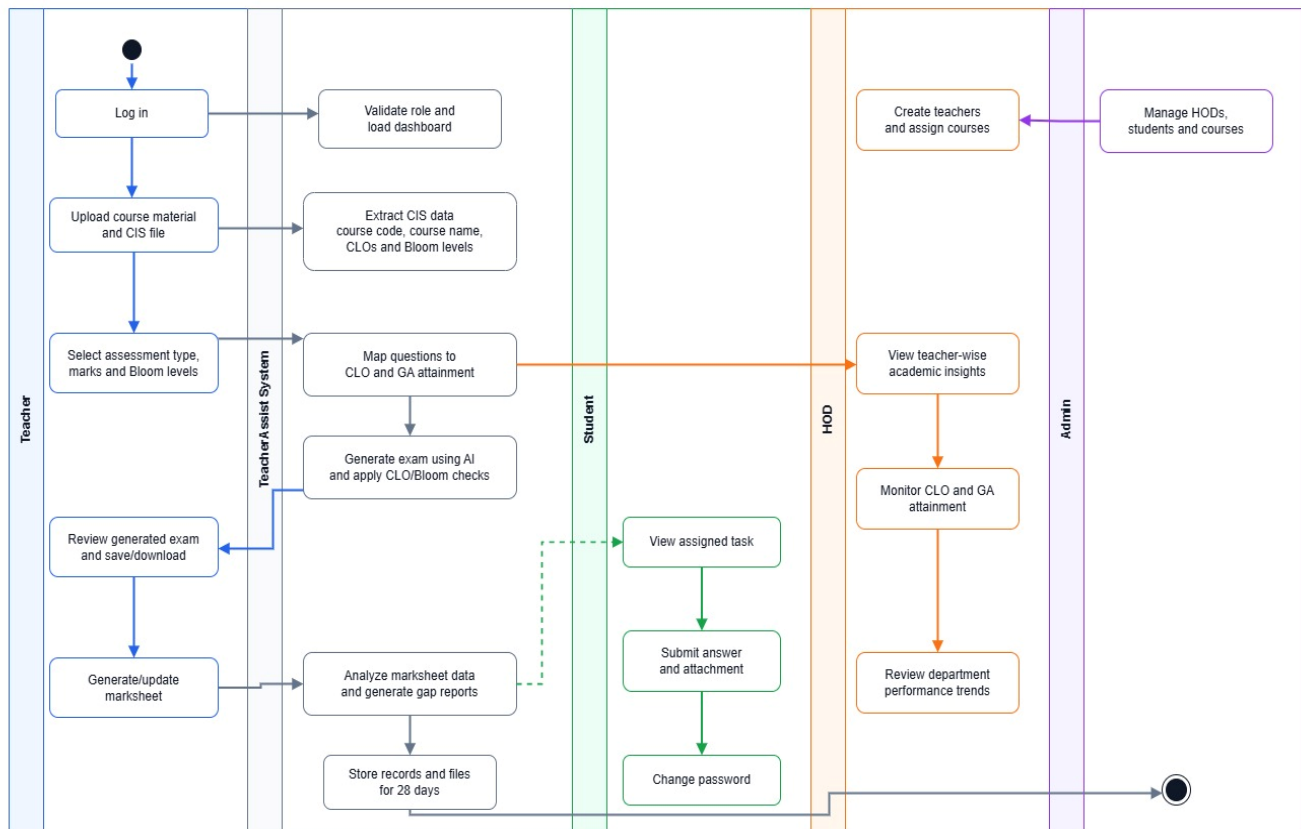


Figure 4. 2 Activity Diagram for Teacher Assist

[Figure 4.2](#) illustrates the flow of activities of Teacher Assist system to map and report automated OBE. The user logs in through the web application and uploads and views course material or the levels of Bloom and requests CLO-GAs mapping. The system verifies the user, processes data with trained models, and generates OBE reports, mappings, and KPI reports which are stored in the database. Lastly, the user will be able to review or edit the reports and thereafter complete the process.

4.4.3. Use case Diagram:

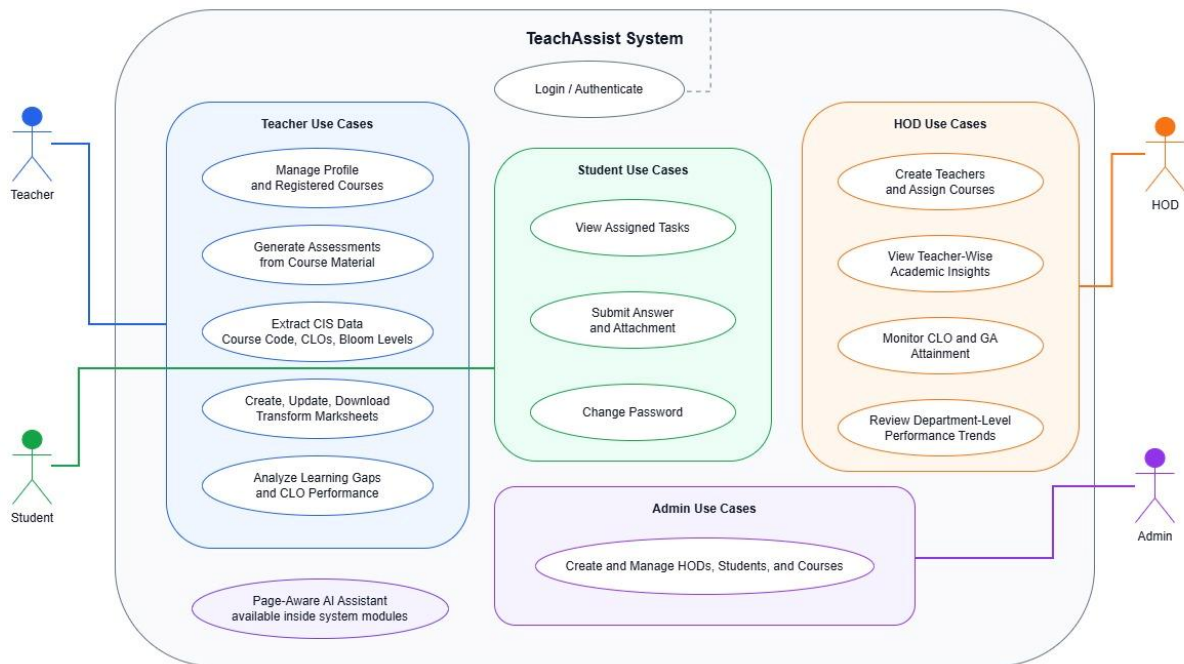


Figure 4. 3 Use Case Diagram for Teacher Assist

In [Figure 4.3](#) a Use Case Diagram for the Teacher Assist system. It depicts the interaction of different users with the system by initially logging in, and then uploading course material. The user may request the NLP based keyword extraction to produce the assessments, perform mapping and finally review or edit the generated reports. In addition to that viewing and generating reports for KPI analysis is also being shown.

CHAPTER 5

5. Software Design and Modelling

Software design and modelling is one of the most significant stages of software development process as it gives a complete blueprint of the system to be implemented before starting coding. At this point, the purpose of the system, its parts, their interactions, data flow and system behavior are well established. It provides the developer with an understanding of the interactions between system parts, and how the intended system functions will be achieved.

The design and modelling of the Teacher Assist system is presented in this chapter by using various UML diagrams like class diagrams, sequence diagrams and object diagrams. These diagrams are used to describe the internal structure of the system, the key modules, the users and the interactions between various parts of the system. The class diagram shows the major classes of Teacher Assist, their attributes, methods and associations, and the object diagram shows the objects that exist at a specific point in time.

Teacher Assist is an AI Academic support and assessment management tool for University Admins, HODs, Teachers, and Students. The system includes features for authenticating and controlling access, setting up teacher profiles, generating assessments, extracting CLOs from CIS, checking for alignment with Bloom and CLO mapping to GA, creation of a transcript in Excel, a page aware AI assistant and more.

Additionally, this chapter describes the behavioral aspects of Teacher Assist including user interaction with the system, as well as the interactions among the various modules. The user login, course management and teacher-course registration are the main functionalities of the system. Uploading of course material and CIS, extraction of course and CLO information, generation of quizzes and exams, transcript generation, analysis of student performance, assignment of tasks to weak students, review of student submissions, transcript updating, and monitoring CLO and GA attainment are all other features of the system.

This chapter aims to give a clear design perspective of the Teacher Assist system prior to its implementation. It describes the organization of the system, relationship between the elements and the contribution of the behavior of each module to the functioning of the whole system. The objective of this design phase is to create a system which is well planned, well organized, maintainable and meets the project requirements.

5.1. Project Architecture:

The Teacher Assist project has been developed using client-server architecture. The system is split into two big parts: Client side, Server side. User interaction is handled on the client side, processing and business logic on the server side, file operations on the server side, database interaction on the server side, and generation on the server side using AI. This division allows the system to be more organized, maintainable and easier to understand.

Angular, HTML, CSS and TypeScript are used for client-side development. Features its own admin, HOD, teacher and student dashboards and interfaces. Only the functions that are allowed to each user role are available. The admin can configure and manage HODs, students and courses. HOD can generate teachers, set courses and view academic insights. The Teacher can leverage the Use the Profile/Setup module, generate module, Transform module, Gap Analysis module, Student Task Review module, and AI Assistant module. The student can see Tasks assigned to them, download reference files, submit answers, add attachments to answers, check Review status, and change password.

The server side is developed using Python, FastAPI, and SQLAlchemy. It is responsible for all the basic functionalities of the system. It encompasses authentication, role checking, uploading of files, text extraction, CIS data extraction, CLO mapping, Bloom mapping, generation of assessments, creation of the transcript, conduct a gap analysis, task assignment, task review, generation of HOD insight, and database management. Frontend sends request to backend, which processes the request based on the selected operation, and sends back the appropriate response.

Teacher Assist's production database is a PostgreSQL database served via Supabase. It saves the user records, roles, courses, teacher assignments, CIS information, assessment records created transcript records, GA information, student tasks, student submissions, review results, CLO attainment, GA attainment, and file references. The uploaded files, generated PDFs, Excel transcripts, teacher attachments, and student submission attachments are managed via backend file storage, and their associated records, file paths are stored in the database.

Teacher Assist leverages Groq API and LLaMA 3.3 70B for AI Text Generation. Teachers upload Course Information Sheet and course material in the Generate module. The system retrieves the Course Code, Course Name, CLO number, CLO description, Bloom's cognitive levels and weekly topics (if available). Data gleaned is utilized to create quizzes, assignments, midterm exams and final exams. The assessment produced contains the Course Code, Course Name, Exam Type, Total Marks, Exam Time, Question Numbering, Mark per Question, Bloom's Level and CLO Tags.

The system maps the questions to CLOs based on CIS CLO descriptions, Bloom level, type of question, topic relevance of uploaded content and OBE-based matching logic. The system will also display a visible Bloom/CLO mismatch warning if the teacher chooses a higher mapped Bloom level than the mapped CLO. This enables the teacher to check the assessment before saving or downloading it.

The architecture also facilitates mapping of CLO to GA for academic monitoring at HOD level. The system is mapped out in a sequence of how questions perform to CLO and how CLO is mapped to GA. This enables the chairperson to have a view of teacher-wise insights, course-wise performance, CLO attainment, GA attainment, and the trends of performance in the department.

Teacher Assist is a modular approach. Main modules include Authentication and Role Based Access, Admin Management, chairperson Management, Teacher Profile/Setup, Generate, Transform, Gap Analysis, Student Task Management, chairperson Insights, AI Assistant, File Storage and Database Management.

For instance, the Generate module can be enhanced without altering the Student Task module. Likewise, the Transform module to create transcripts can be changed without impacting the Gap Analysis module. Future replacement and upgrading of the AI model without changing the complete frontend interface.

In summary, the chosen architecture provides Teacher Assist scalability, flexibility and maintenance ease. It is deployed on a Vercel frontend, a Render Web Service backend, and a Supabase hosted PostgreSQL database. With this deployment, the system can be accessed via the internet and the logical separation of the frontend, backend, database, AI service, and file handling.

5.2. Client–Server Layered Architecture:

Teacher Assist is a layered (client-server) application. This is an architecture that splits the system into various layers which explicitly have their own responsibilities. The main layers of the system are the Presentation Layer, Application Layer, AI Processing Layer, Data Access Layer, Database Layer and File Storage Layer.

The Presentation Layer is the “front end” of the system. Built with Angular, HTML, CSS and TypeScript. This layer is used for the GUI of Admins, HODs, Teachers and Students. It is the layer on which users log in, manage records, upload files, configure assessments, create transcripts, view gap analysis, assign tasks to users, submit answers, view dashboards, etc.

The FastAPI backend deals with the Application Layer. It is an interface that takes data from the front end and governs the primary business logic of the system. This layer validates user roles, validates inputs, controls workflows, calls services when necessary, and returns responses to the front-end. By example, if a teacher uploads the course content to the server, then CIS to generate the assessments, the Application Layer will handle the entire process of receiving files and generating assessments.

The AI Processing Layer is responsible for intelligent operations. It leverages Groq LLaMA 3.3 70B for text-based generation and AI assistance. This layer aids in generating quizzes, assignments, midterm exams and final exams from uploaded course material and CIS information in the Generate module. It assists in answering user queries with respect to the current page and visible records, and available system modules, in the AI Assistant module.

The Data Access Layer communicates between the Backend and Database. SQLAlchemy is used to manage database models and queries. This layer is used to store, update, retrieve, and delete records like users, courses, teacher assignments, generated exams, transcripts, gap analysis results, students’ tasks, students submissions, and chairperson insight records.

Supabase is the host for a PostgreSQL database that is used by the Database Layer. It contains structured system data and file references. It has the capability of role-based access, course management, teacher-course registration, assessment records, transcript data, CLO performance, GA performance, retention related information, task records, and submission records.

Files uploaded and generated are managed by the File Storage Layer. These files consist of uploaded course material, CIS files, generated PDFs, Excel transcripts, teacher reference attachments and student submission attachments. The file references are stored in the database, and the backend is responsible for handling the files. Transform transcripts and student task records/files are retained for 28 days. These records, along with related files, are automatically removed after 28 days.

The multi-layer design contributes to maintaining system organization and simplicity in system maintenance. If one layer is to be updated, the other layers can be mostly unchanged. For instance, the front-end look and feel may be enhanced without modifying the database. Likewise, the AI provider can be upgraded in the future, without altering the overall system structure.

Overall, the client-server layered architecture is a solid foundation for Teacher Assist. It splits the user interaction, the backend processing, the AI generation, the database management, and the file storage into different layers. This provides greater reliability, scalability, testability and future improvement in the system.

5.2. UML Diagrams

This section includes the following UML diagrams of SAAS web application. It consists of three main modules:

1. Generation Module
2. Transformation Module
3. Gap Analysis

5.2.1. Class Diagram

A class diagram is a UML diagram which expresses the static structure of a software system. It displays the main classes, attributes, methods and relationships. Teacher Assist uses class diagrams to illustrate the relationships between the key objects in each module, and how data flows between teachers, courses, assessments, transcripts, students, CLOs, and reports.

5.2.1.1. Generation Module:

The class structure for the Generate module in Teacher Assist is shown in Figure 5.3.1.1. The teacher uploads course material and a Course Information Sheet in this module. The system retrieves the Course Code, Course Name, CLO numbers, CLO descriptions, Bloom's taxonomy levels if available.

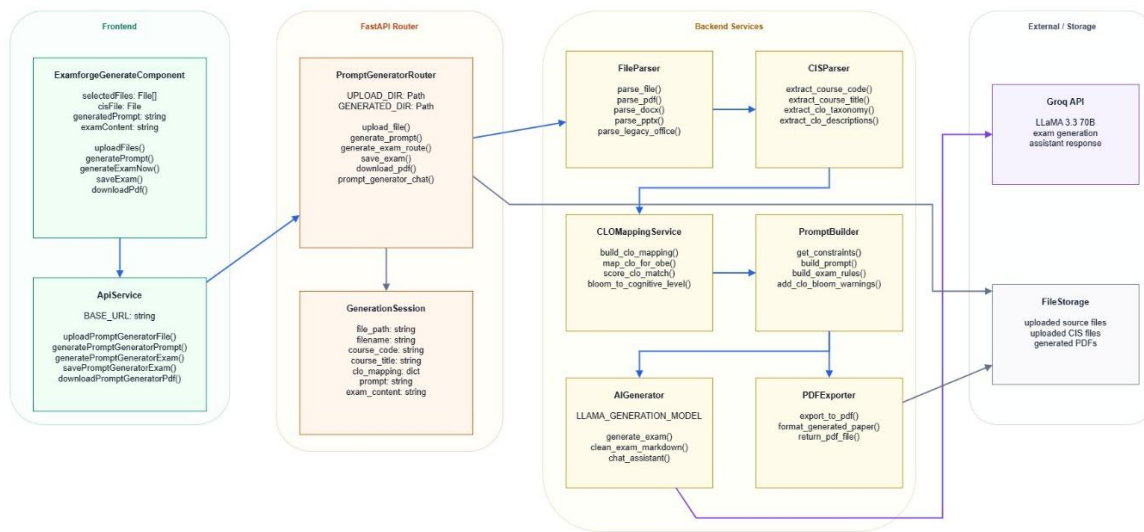


Figure 5.1: Generation Module of Teacher Assist

5.2.1.2. Gap Analysis Module:

[Figure 5.2](#) represents the Gap Analysis module of Teacher Assist, the system computes CLO performance, generates KPI-based OBE reports and provides targeted learning recommendations to support academic improvement.

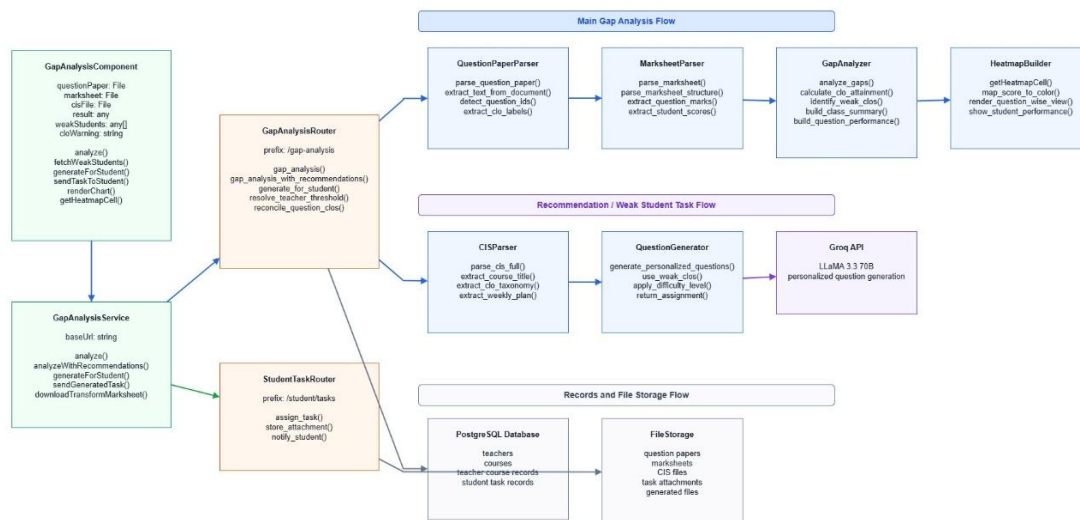


Figure 5.2: Gap Analysis Module of Teacher Assist

5.2.1.3. Transformation Module:

Figure 5.3 illustrates the Document Transformation module of Teacher Assist, which extracts documents, maps the semantics to a target document template format, and maps the document to the target document template. The system handles various types of files, maps them intelligently into columns and exports the transformed data to the desired output format with the same data structure.

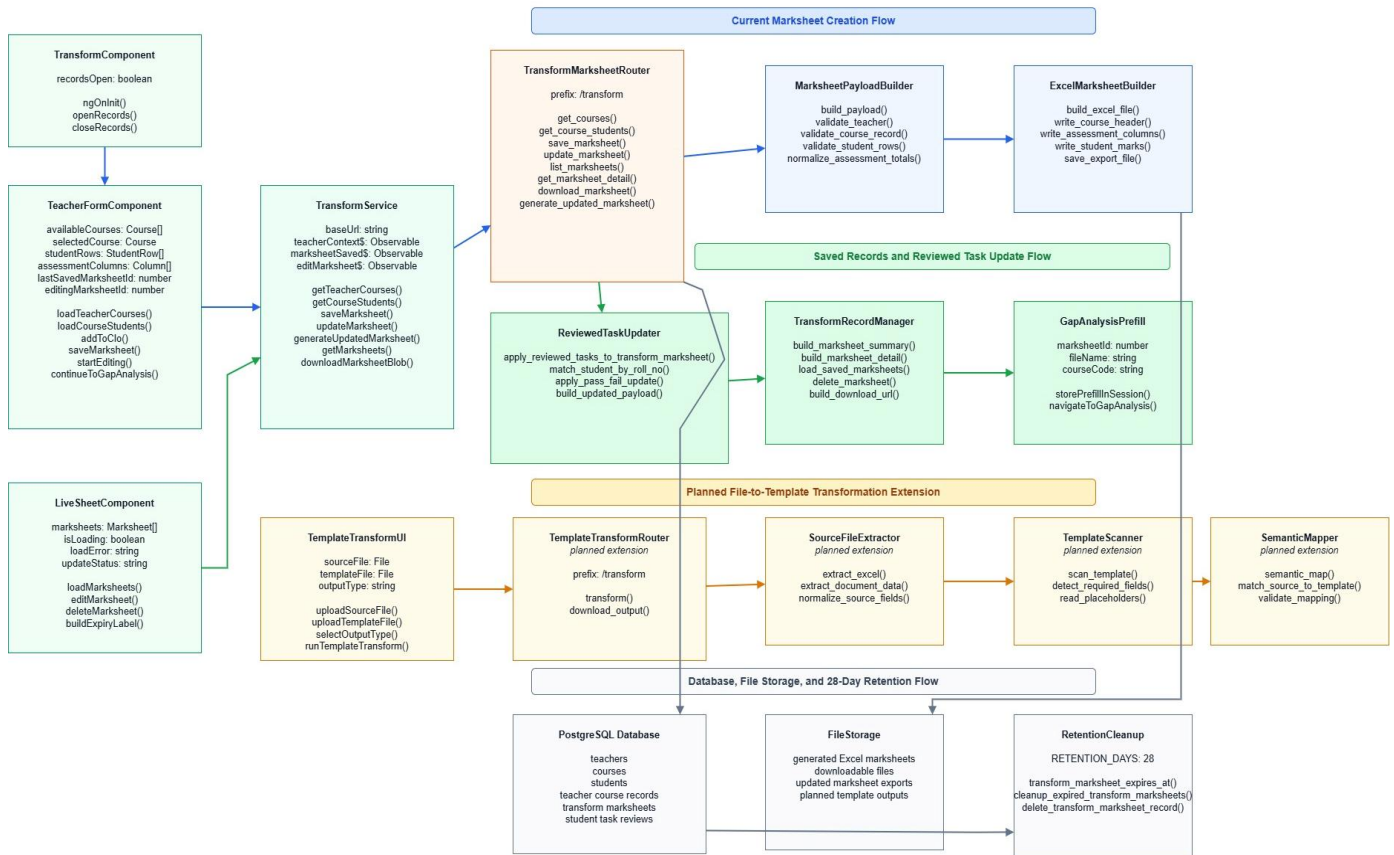


Figure 5.3: Transformation Module of Teacher Assist

5.2.2. Sequence Diagram

Sequence diagram is a UML diagram which visualizes flow of data between objects which starts from the first action and ends with the final action performed.

5.2.2.1. Generation Module:

Figure shows an example of the workflow for creating an assessment. The system reads uploaded documents, retrieves relevant context with the RAG engine, and creates quizzes, assignments or exams with the LLM, based on selected CLOs and Bloom's levels.

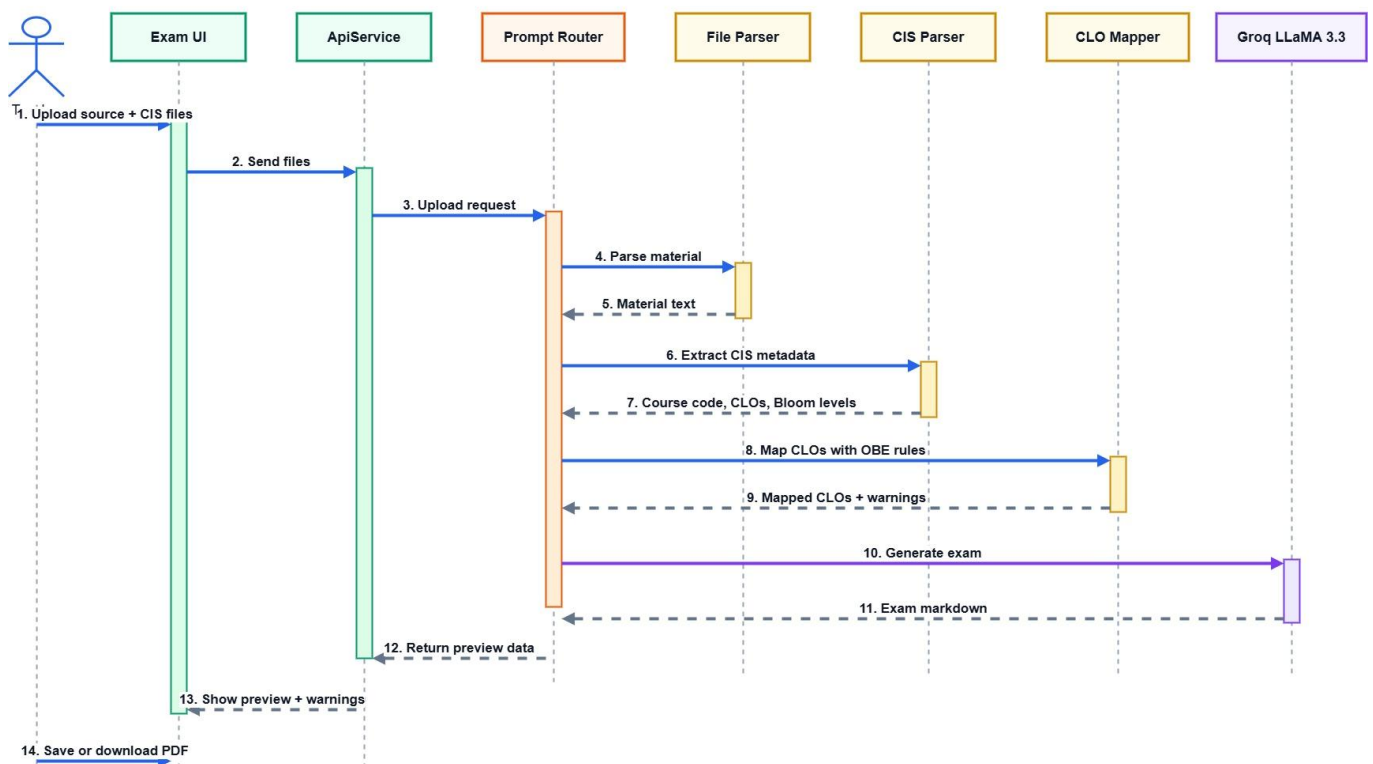


Figure 5.4: Generation Module of Teacher Assist

5.2.2.2. Gap Analysis Module:

[Figure 5.5](#). the sequence diagram shows the Teacher Assist workflow of gap analysis. It uploads and processes the exam results and question papers, uses a predetermined threshold to analyse the performance at CLO level and by questions, and produces gap analysis report detailing the learning gaps.

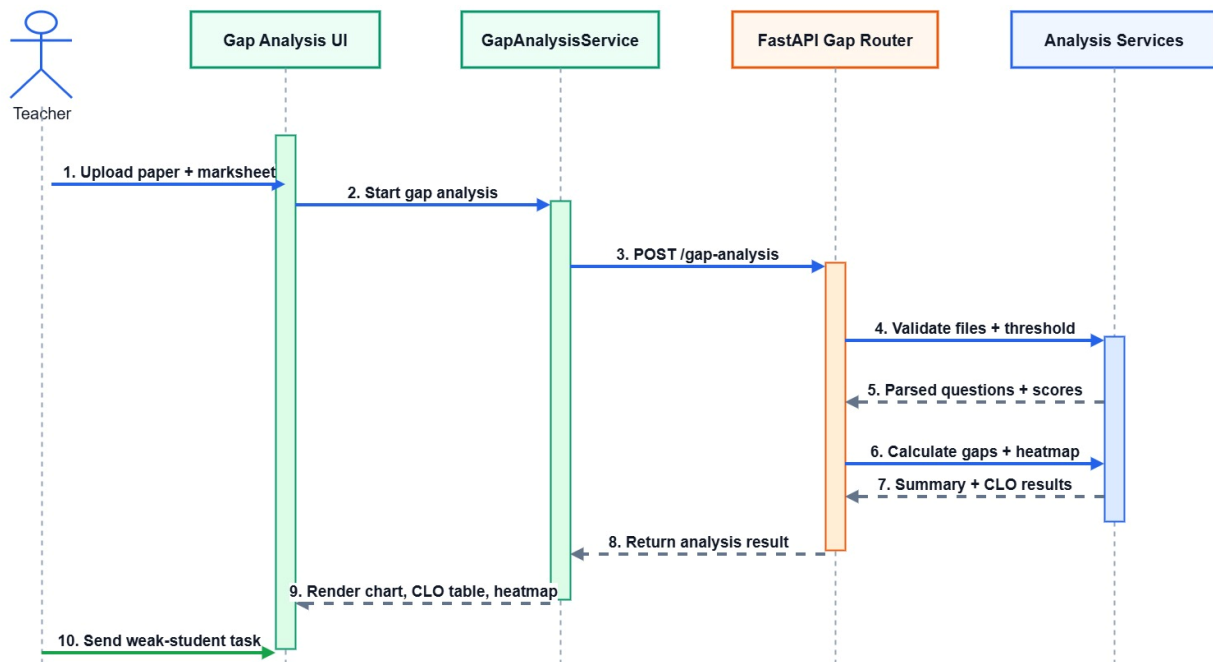


Figure 5.5: Gap Analysis Module of Teacher Assist

5.2.2.3. Transformation:

This sequence diagram shows how Teacher Assist transforms documents using an AI-based template approach. The system extracts data from the source and template files, semantically maps the content, and generates a transformed document in the selected format for the teacher to download.

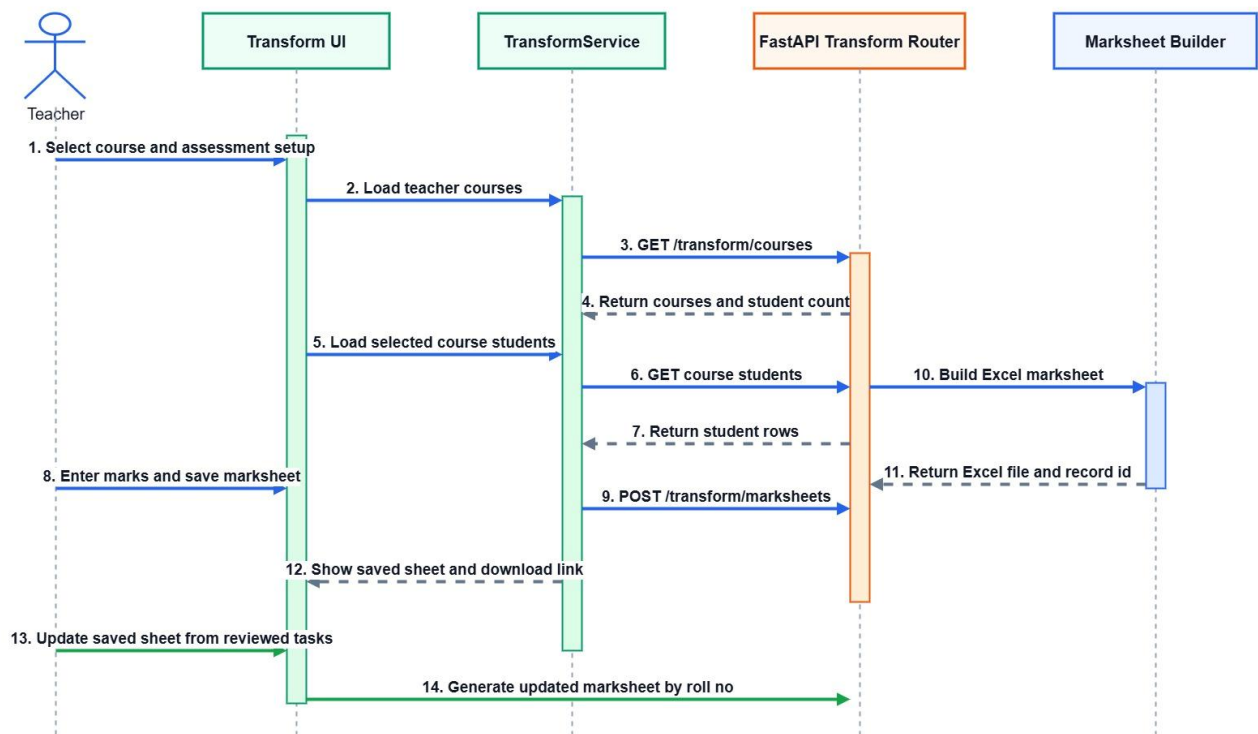


Figure 5.6: Transformation Module of Teacher Assist

5.2.3. Object Diagram:

5.2.3.1. Generation Module:

This object diagram shows the runtime interaction of objects in the Quiz, Assignment, Mid, and Final generation module. The teacher uploads course and CIS documents select assessment parameters, and the system extracts content, retrieves context via RAG, and generates assessment through the LLM while ensuring CLO and Blooms alignment.

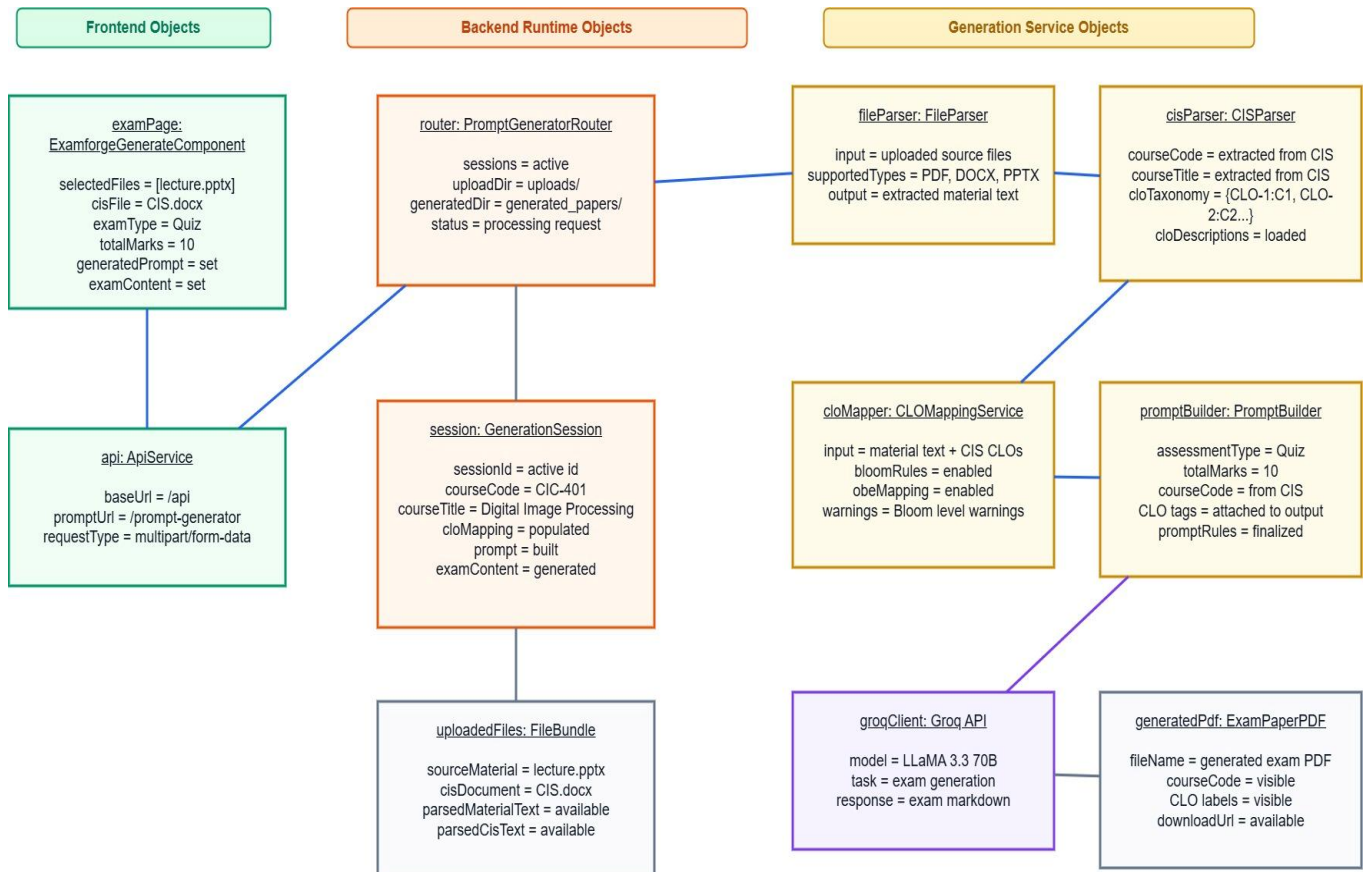


Figure 5.7: Generation Module of Teacher Assist

5.2.3.2. Gap Analysis Module:

This object diagram shows how assessment results are analyzed against a predefined threshold to evaluate CLO performance. The system identifies weak CLOs using KPI analysis and generates improvement recommendations, which are then viewed by the teacher.

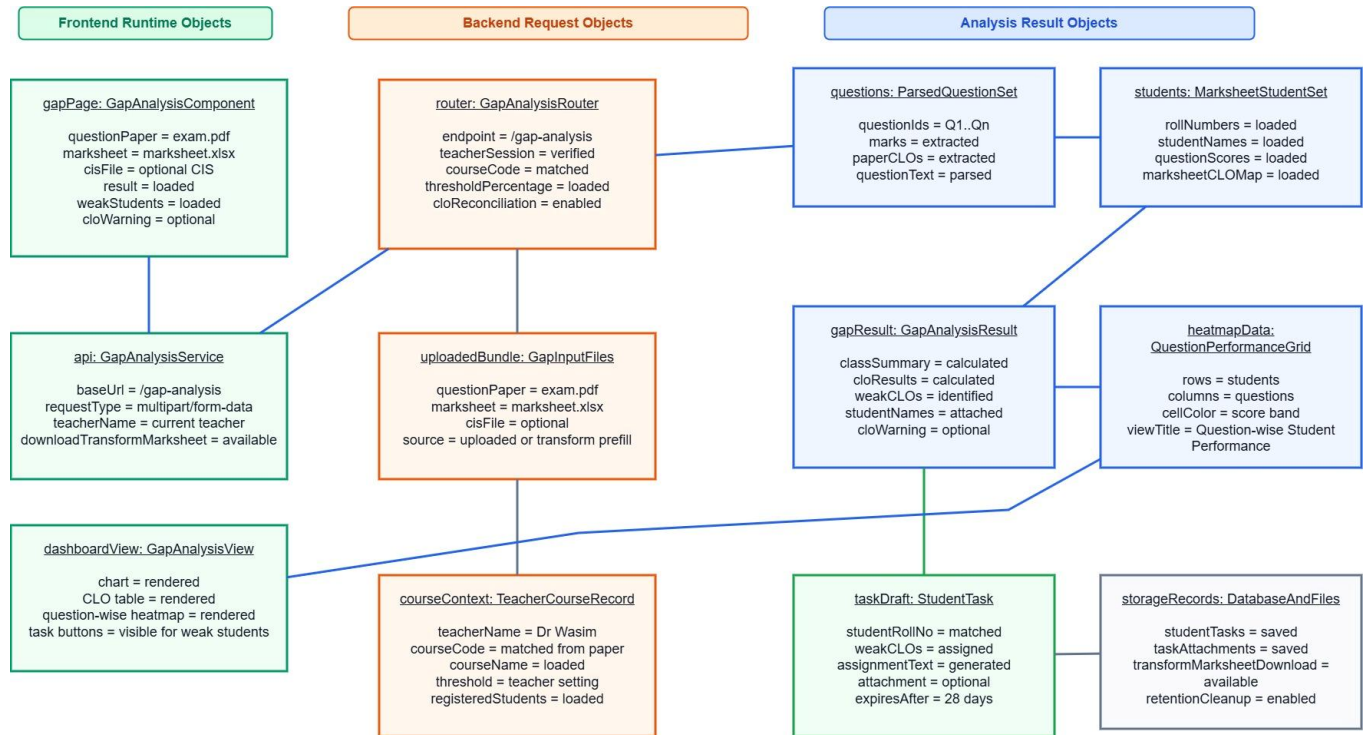


Figure 5.8: Gap Analysis of Teacher Assist

5.2.3.3. Transformation Module:

This object diagram represents the document transformation process where source files are extracted, semantically mapped, and applied to a target template. The transformed output is then exported into the selected format while preserving the original data structure.

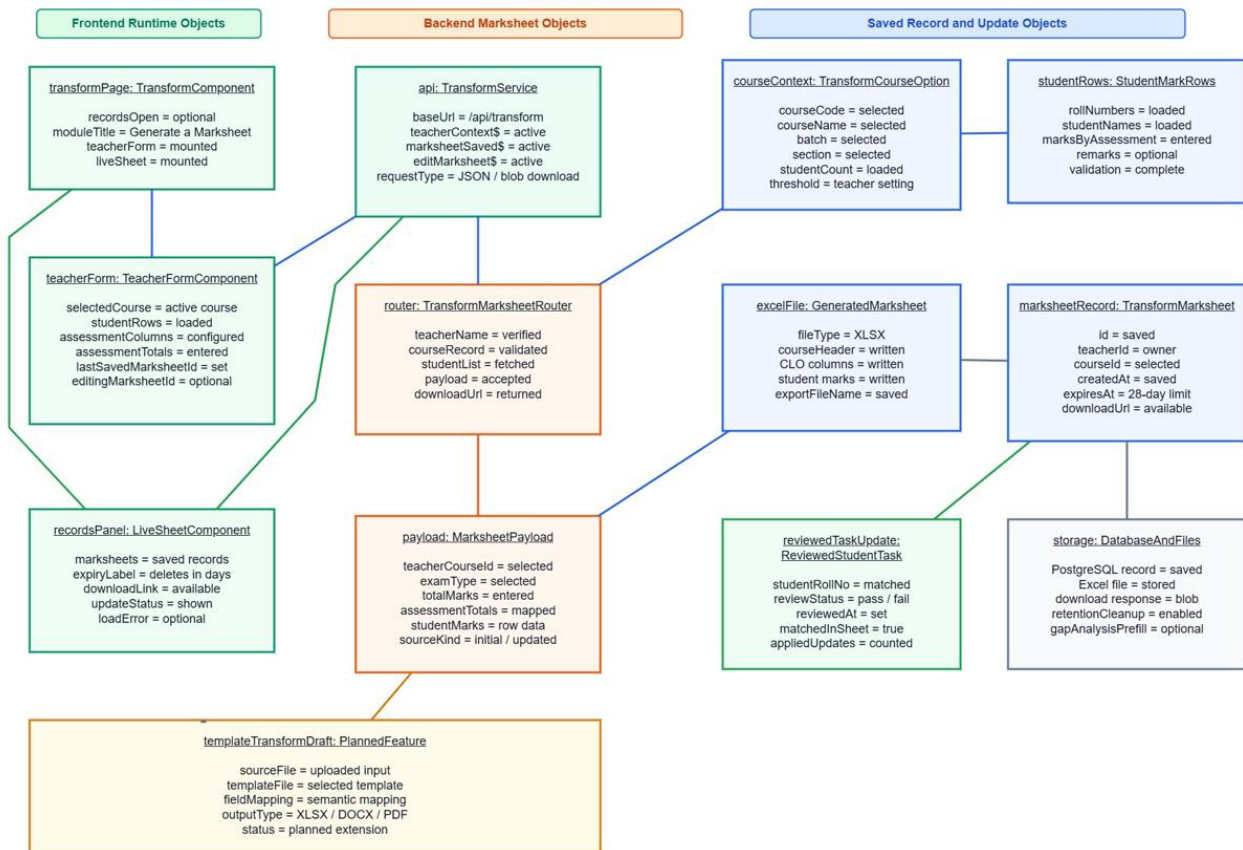
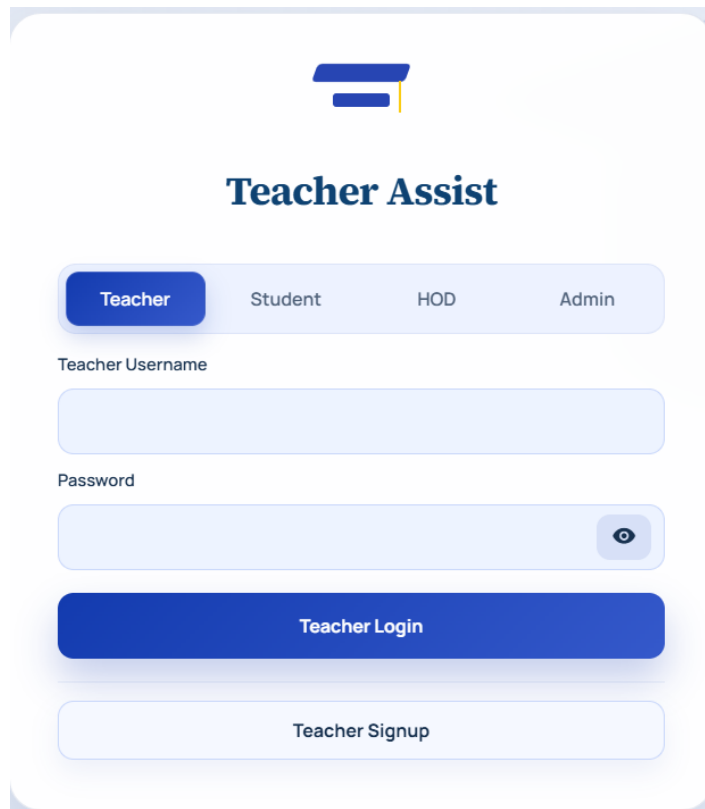


Figure 5.9: Transformation of Teacher Assist

5.3. Interface:

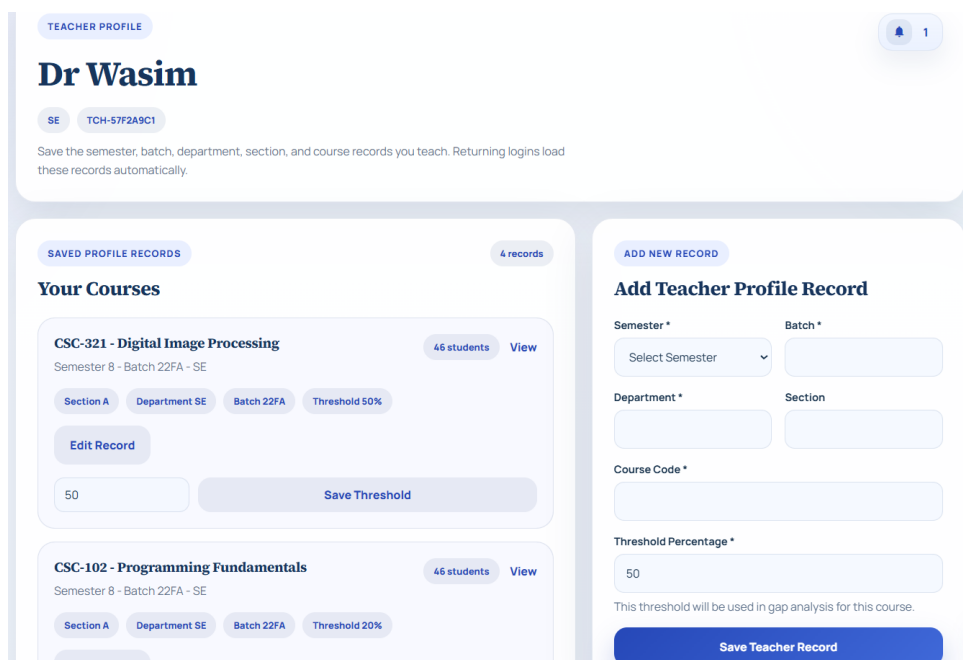
[Figure 5.10](#) shows the login page of Teacher Assist, where users can access the system according to their roles such as teacher, student, HOD, or admin.



The login interface for Teacher Assist features a central card with a blue header and a logo consisting of two horizontal bars. Below the header, there are four role-based tabs: 'Teacher' (highlighted in blue), 'Student', 'HOD', and 'Admin'. Under the 'Teacher' tab, there are two input fields: 'Teacher Username' and 'Password' (with a toggle for visibility). Below these fields are two buttons: a prominent blue 'Teacher Login' button and a lighter blue 'Teacher Signup' button.

Figure 5.10: Teacher Assist Login Interface

[Figure 5.11](#) shows the teacher profile setup page where teacher information, assigned courses, batch, section, and threshold percentage can be managed.



The profile setup page for Dr. Wasim (SE, TCH-57F2A9C1) is divided into two main sections. The left section, titled 'Your Courses', lists two courses: 'CSC-321 - Digital Image Processing' and 'CSC-102 - Programming Fundamentals'. Each course entry shows semester, batch, section, department, and a threshold percentage, with buttons for 'View', 'Edit Record', and 'Save Threshold'. The right section, titled 'Add Teacher Profile Record', contains a form with fields for Semester, Batch, Department, Section, Course Code, and Threshold Percentage, followed by a 'Save Teacher Record' button.

Figure 5.11: Profile Setup for Teacher Assist

[Figure 5.12](#) shows the document upload step where teachers can upload lecture files and Course Information Sheet files. The uploaded files are used for content extraction, CLO identification, and assessment generation.

Figure 5.12: Document Upload Interface for Exam Generation

[Figure 5.13](#) shows the exam configuration page where teachers select exam type, question count, marks, and Bloom's level.

Figure 5.13: Exam Configuration Interface

Figure 5.14 shows the generated prompt page where the system prepares an AI prompt based on uploaded documents and exam settings. The teacher can review, copy, save, or proceed to generate the exam paper.

Figure 5.14: AI Prompt Generation Interface

Figure 5.15 shows the editable exam paper interface where teachers can review and modify the generated assessment before saving. It also provides options to save or download the final paper in PDF format.

Figure 5.15: Generated Exam Editing and Saving Interface

[Figure 5.16](#) shows the transformation module used to generate a transcript for a selected course. It allows teachers to define CLOs, assessment type, marks, and student marks entry for OBE-based evaluation.

The screenshot shows the 'TRANSFORM MODULE' header with a 'View Records' button. The main title is 'Generate a Marksheet'. The interface is divided into two main sections: 'Course Selection' and 'Dropdown Columns'. The 'Course Selection' section includes fields for Teacher (Dr Wasim), UID (TCH-57F249C1), Select Course (a dropdown menu), Department (SE), Semester (Semester B), Batch (22FA), Section (A), Total Marks (100), and Excel File Name (CSC-321_22FA_A_marksheet). The 'Dropdown Columns' section includes 'Select CLO' (a dropdown menu), 'Assessment Type' (a dropdown menu), 'Assessment Total Marks' (a text input field), and an 'Add To CLO' button. At the bottom, there is a 'Student Marks Entry' section with a message 'No students loaded yet. Select one of your registered courses first.' and a 'Save Marksheet' button.

Figure 5.16: Transcript Generation Interface

[Figure 5.17](#) shows the gap analysis module where teachers upload the question paper and transcript files. The system uses these files to identify weak CLOs and student performance gaps.

The screenshot shows the 'Gap Analysis - TeachAssist' interface. It features two main upload buttons: 'Upload question paper' (PDF / DOCX) and 'Upload marksheet' (Excel / CSV). Below these buttons is an 'Analyze Gap' button.

Figure 5.17: Gap Analysis Upload Interface

[Figure 5.18](#) shows the gap analysis result after processing the uploaded question paper and transcript. It displays the selected course, threshold percentage, and next steps for loading weak students.

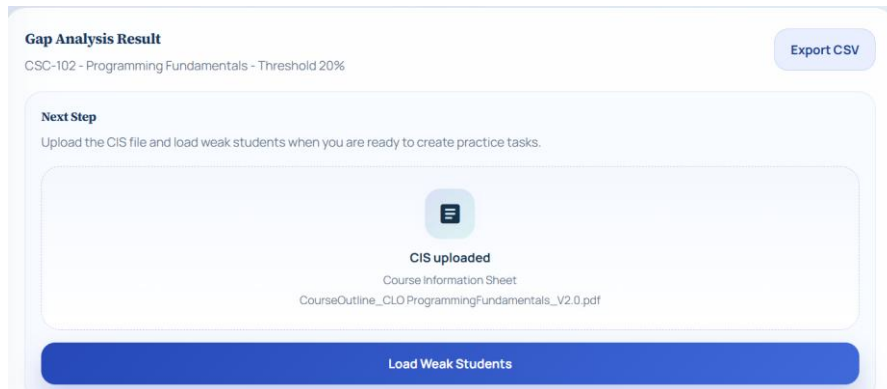


Figure 5.18: Gap Analysis Result Interface

[Figure 5.19](#) shows a graphical representation of passed and below-threshold percentages for each question mapped with CLOs. The bar chart helps teachers quickly understand class performance across different CLOs.

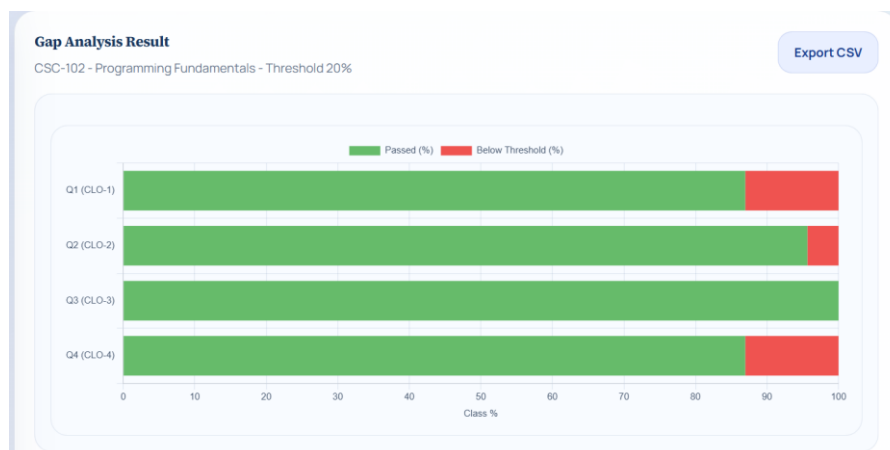


Figure 5.19: CLO Performance Chart

Figure 5.20 shows question-wise CLO gap analysis including max marks, threshold marks, students below threshold, gap percentage, and status. It helps teachers identify which questions and CLOs need improvement.

QUESTION	CLO	MAX MARKS	THRESHOLD MARKS	STUDENTS BELOW	GAP %	STUDENT NAMES	STATUS
Q1	CLO-1	5	1	6	13.04%	Hamza Ahmed Siddiqui, Khubaib Raza, MAHAD AHMED ABBASI, Muhammad Ahsan, Muhammad Huzaifa, Usman Habib	Gap Identified
Q2	CLO-2	5	1	2	4.35%	Ehsan Afzal, MAHAD AHMED ABBASI	Gap Identified
Q3	CLO-3	5	1	0	0%	-	No Gap
Q4	CLO-4	5	1	6	13.04%	Ahmed Muarij Siddiqui, alyan uddin, Areesha Feroz, Mannan ali, Muhammad Ahsan, Roha Pathan	Gap Identified

Figure 5.20: Question-Wise Gap Analysis Table

Figure 5.21 presents a CLO-wise summary of weak and achieved CLOs based on student performance. It supports teachers in identifying learning outcome gaps for further academic action

CLO-wise Gap Analysis						
CLO	QUESTIONS	MAX MARKS	THRESHOLD MARKS	STUDENTS BELOW	GAP %	STATUS
CLO-1	Q1	5	1	6	13.04%	Weak CLO
CLO-4	Q4	5	1	6	13.04%	Weak CLO
CLO-2	Q2	5	1	2	4.35%	Weak CLO
CLO-3	Q3	5	1	0	0%	OK

Figure 5.21: CLO-Wise Gap Analysis Summary

Figure 5.22 shows the list of students who performed below the required threshold in specific CLOs. Teachers can use this information to generate targeted practice tasks for weak students.

Students Requiring Attention				Download PDF
#	STUDENT NAME	WEAK CLOS	ACTION	
1	Ahmed Muarij Siddiqui	CLO-4	Generate	
2	alyan uddin	CLO-4	Generate	
3	Areesha Feroz	CLO-4	Generate	
4	Ehsan Afzal	CLO-2	Generate	
5	Hamza Ahmed Siddiqui	CLO-1	Generate	
6	Khubaib Raza	CLO-1	Generate	
7	MAHAD AHMED ABBASI	CLO-1 CLO-2	Generate	

Figure 5.22: Students Requiring Attention Interface

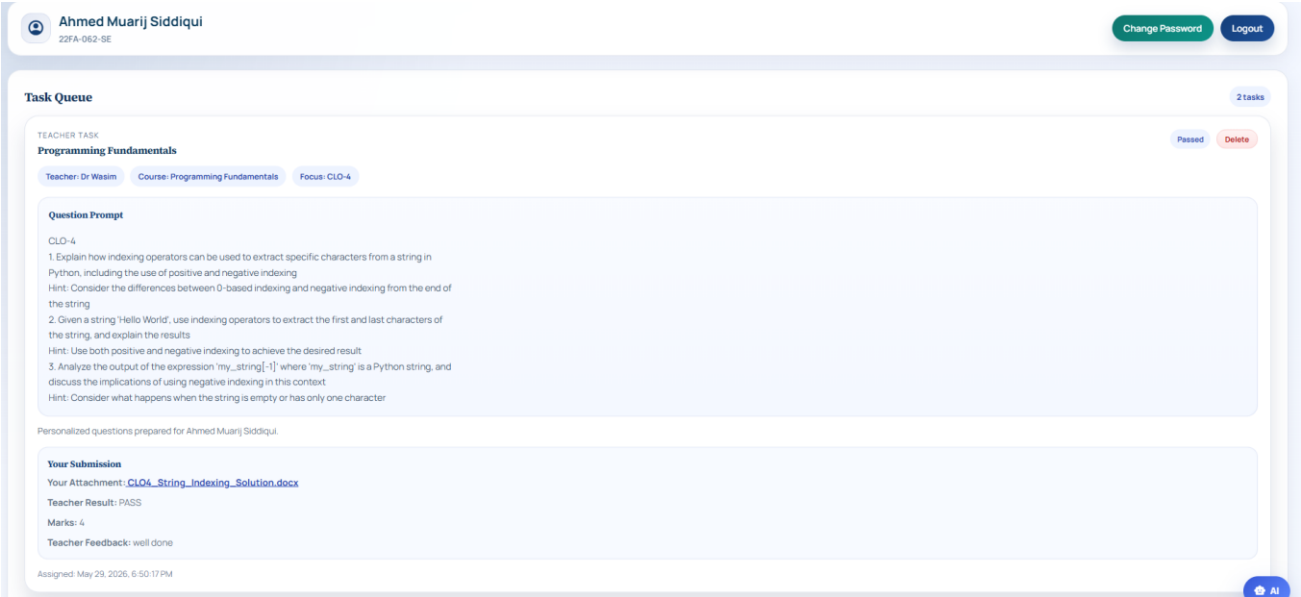
[Figure 5.23](#) shows the student login page of Teacher Assist, where students can access the system using their student ID and password. The role-based login interface ensures that students are directed to their own dashboard after authentication.



The image shows a login interface for 'Teacher Assist'. At the top, there is a logo consisting of two horizontal bars, one blue and one yellow. Below the logo, the text 'Teacher Assist' is displayed in a bold, dark blue font. Underneath, there are four tabs: 'Teacher', 'Student' (which is highlighted with a dark blue background), 'HOD', and 'Admin'. Below the tabs, there are two input fields: 'Student ID' with the value '22fa-062-se' and 'Password' with a masked input (dots). To the right of the password field are two icons: an eye and a lock. At the bottom, there is a large blue button labeled 'Student Login'.

Figure 5.23: Student Login Interface of Teacher Assist

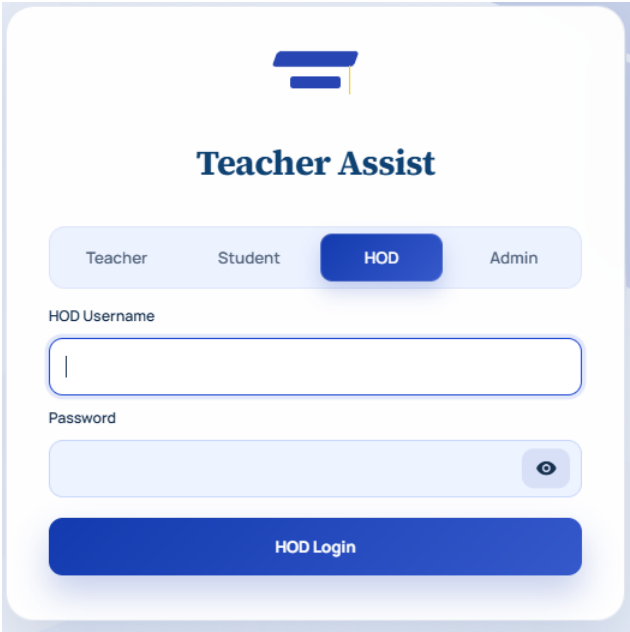
[Figure 5.24](#) shows the student dashboard where assigned practice tasks are displayed with submission and teacher feedback details. It helps students view their weak CLO tasks and track their performance improvement.



The image shows a student dashboard for 'Ahmed Muarij Siddiqui' (22FA-062-SE). At the top right, there are buttons for 'Change Password' and 'Logout'. The main section is titled 'Task Queue' and shows '2 tasks'. The first task is 'Programming Fundamentals' by 'Teacher: Dr. Wasim', 'Course: Programming Fundamentals', and 'Focus: CLO-4'. It includes a 'Question Prompt' for 'CLO-4' with three questions about string indexing in Python. Below the prompt, it shows 'Your Submission' with an attachment 'CLO4_String_Indexing_Solution.docx', a 'Teacher Result: PASS', 'Marks: 4', and 'Teacher Feedback: well done'. At the bottom, it says 'Assigned: May 29, 2025, 6:50:17 PM'. There is also an 'AI' icon in the bottom right corner.

Figure 5.24: Student Task Queue Interface

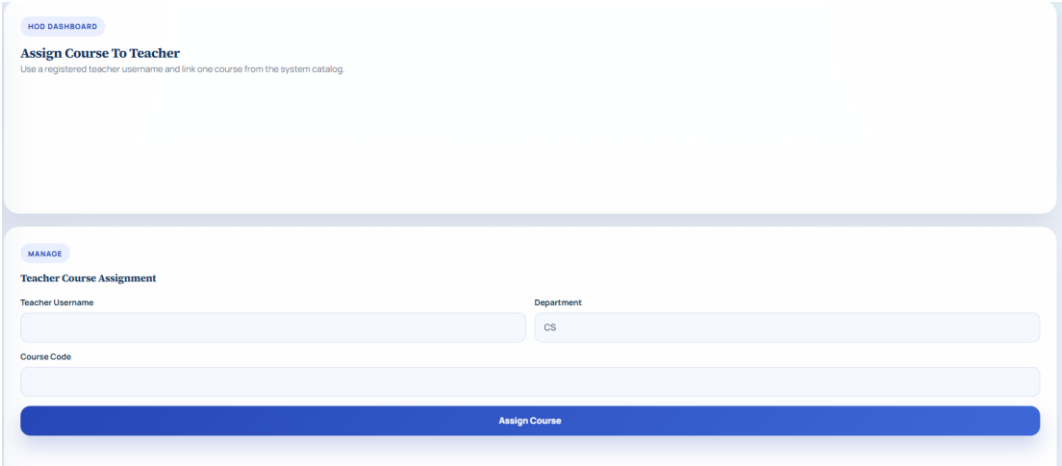
Figure 5.25 shows the HOD login page of Teacher Assist, where the Head of Department can access the system using a username and password.



The image shows the HOD login interface of the Teacher Assist system. At the top, there is a logo consisting of two horizontal blue bars. Below the logo, the text "Teacher Assist" is displayed in a bold, dark blue font. Underneath, there are four tabs: "Teacher", "Student", "HOD", and "Admin". The "HOD" tab is currently selected and highlighted in blue. Below the tabs, there are two input fields: "HOD Username" and "Password". The "HOD Username" field is a simple text input. The "Password" field is a text input with a blue eye icon on the right side, indicating a toggle for password visibility. At the bottom of the login section, there is a large blue button labeled "HOD Login".

Figure 5.25: Chairperson Login Interface of Teacher Assist

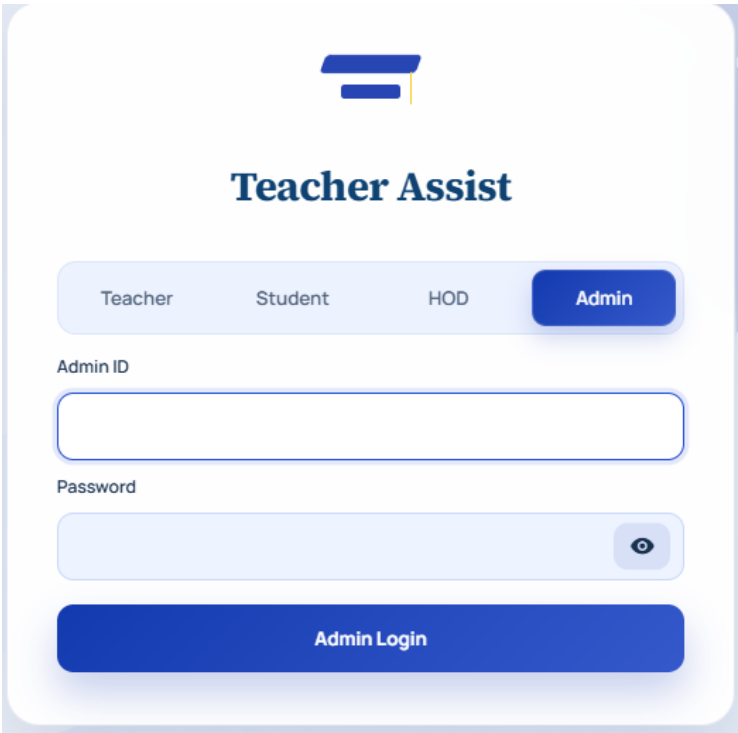
Figure 5.26 shows the HOD dashboard used to assign courses to registered teachers. It supports departmental management by linking teachers with specific courses and departments.



The image shows the HOD dashboard for assigning courses to teachers. At the top, there is a header "HOD DASHBOARD" in a small blue box. Below it, the main heading is "Assign Course To Teacher" in bold, followed by a subtitle "Use a registered teacher username and link one course from the system catalog." in a smaller font. The dashboard is divided into two main sections. The top section is titled "MANAGE" in a small blue box. Below it, the heading "Teacher Course Assignment" is displayed. Under this heading, there are three input fields: "Teacher Username", "Department", and "Course Code". The "Department" field is currently set to "CS". At the bottom of the form, there is a large blue button labeled "Assign Course".

Figure 5.26: Chairperson Course Assignment Interface

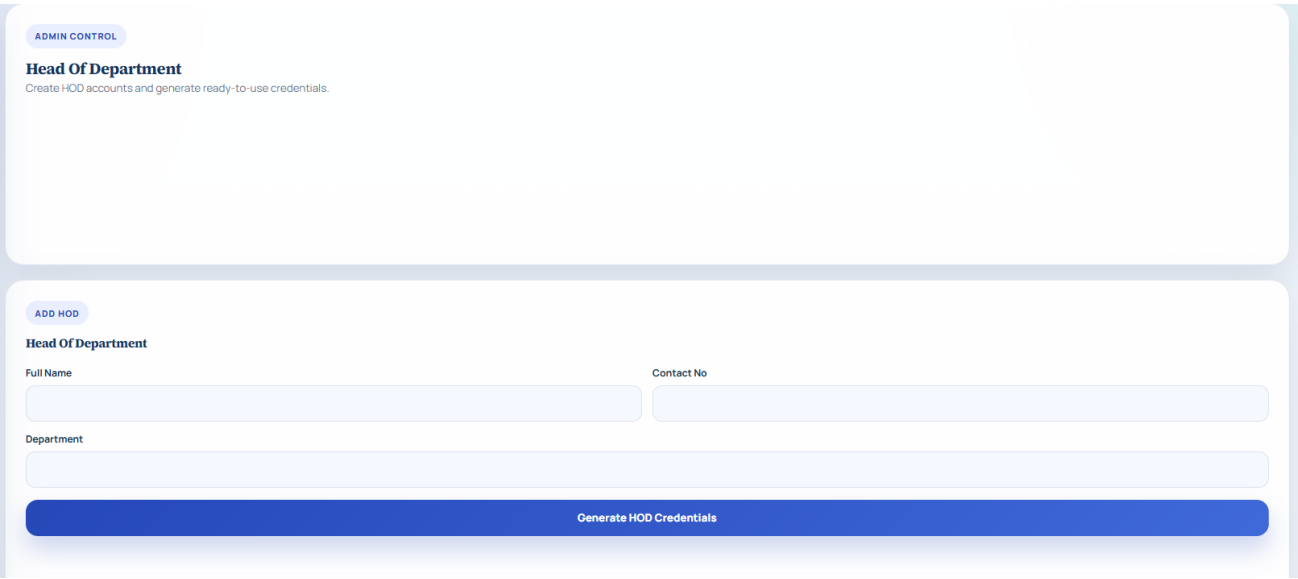
Figure 5.27 shows the admin login page of Teacher Assist. It allows administrative users to access system-level management features securely.



The image shows a login interface for 'Teacher Assist'. At the top, there is a logo consisting of two horizontal bars, one blue and one yellow. Below the logo, the text 'Teacher Assist' is displayed in a bold, dark blue font. Underneath, there are four tabs: 'Teacher', 'Student', 'HOD', and 'Admin'. The 'Admin' tab is highlighted with a blue background. Below the tabs, there are two input fields: 'Admin ID' and 'Password'. The 'Password' field has a toggle icon (an eye) to its right. At the bottom, there is a large blue button labeled 'Admin Login'.

Figure 5.27: Admin Login Interface

Figure 5.28 shows the admin control panel for creating Head of Department accounts. The admin can enter HOD details and generate ready-to-use credentials for departmental access.



The image shows an 'ADMIN CONTROL' panel. At the top, there is a section titled 'Head Of Department' with a subtitle 'Create HOD accounts and generate ready-to-use credentials.' Below this, there is a section titled 'ADD HOD' with a subtitle 'Head Of Department'. This section contains three input fields: 'Full Name', 'Contact No', and 'Department'. At the bottom of this section, there is a large blue button labeled 'Generate HOD Credentials'.

Figure 5.28: Chairperson Account Creation Interface

Figure 5.29 shows the admin panel for registering students either manually or by importing an Excel sheet. The generated student credentials are saved in the system for login and academic access.

ADMIN CONTROL

Student Access
Register students, save them in the database, and prepare their login details.

ADD STUDENT

Student Access

Import Students From Excel
Upload an .xlsx sheet and TeachAssist will save students in the main table and in department-specific tables such as 'students_se_22fa'.

Choose Excel Sheet **Import Students**

Full Name

Roll No

Contact No

Generated Email

Department

Program

Batch

Section

Semester

Generate Student Credentials **AI**

Figure 5.29: Student Access Management Interface

Figure 5.30 shows the admin interface for adding courses to the central course catalog. The course code and course name are stored so they can later be assigned to teachers and used in OBE workflows.

ADMIN CONTROL

Course Catalog
Add courses to the central catalog using course code and course name.

ADD COURSE

Course Catalog

Course Code

Course Name

Save Course

Figure 5.30: Course Catalog Management Interface

CHAPTER 6

6. Algorithm Analysis and Complexity

6.1. Algorithm

An algorithm is a step-by-step procedure that is clear and unambiguous and can be used to solve a problem and/or perform a task. It gives the computer instructions on how to do something, how to do it, and in what order to do it to get the desired results, similar to a recipe for cooking food, or directions to get to a place.

Algorithms are important because they enable the problem-solving process to be systematic and organized. Each step is well defined, and the computer can carry out the step in the same way each time, so that the results are consistent and reliable. Algorithms also facilitate tasks that are more efficient, repeatable and accurate. Algorithms also help to divide up large or difficult problems into manageable steps. This enables the programmers to better understand the problem, design a solution, discover errors, and optimize the performance of a program.

6.2. Pseudo Code

Pseudo code is a description in plain language of the steps in an algorithm. It is written in a style reminiscent of programming codes but is not a strict subset of any one programming language's syntax rules. Pseudo code is not about the correct format of the code, but rather the logic and flow of a solution, written in simple and understandable terms.

Pseudo code is extremely useful as it allows the programmer to plan a program before attempting to write the code for it. It is breaking the solution into clearly ordered steps and makes the problem easier to solve and understand. It has simple writing style, so both programmers and non-programmers can easily understand. It also aids in the early detection of logical errors prior to the program being run in an actual programming language. In software development, pseudo code is frequently used to convey ideas, design algorithms and to step through a program.

6.2.1. Pseudocode of Generation Module:

6.2.1.1. Input:

- Course Material
- Course Information Sheet
- Assessment Type
- Total Marks
- Question Configuration
- Selected Bloom Level

6.2.1.2. Output:

- CLO-mapped Generated Assessment
- Bloom/CLO Warning, if applicable
- Downloadable PDF

6.2.1.3. Begin:

1. Receive course material uploaded by the teacher.

2. Receive the Course Information Sheet uploaded by the teacher.
3. Extract textual content from the uploaded course material.
4. Extract course code, course name, CLO descriptions, and Bloom taxonomy levels from the CIS.
5. Accept assessment configuration from the teacher, including:
 - a. Assessment type, such as Quiz, Assignment, Midterm, or Final
 - b. Total marks
 - c. Question type and number of questions
 - d. Selected Bloom level.
6. Match the uploaded course material with the extracted CIS CLOs.
7. Map each generated question or section to the most suitable CLO.
8. Check whether the selected Bloom level is appropriate according to the CIS CLO taxonomy.
9. If the selected Bloom level is higher than the CLO level, generate a warning.
10. Construct the AI prompt using:
 - a. Extracted course material
 - b. CIS course code and course name
 - c. CLO descriptions
 - d. Bloom taxonomy level
 - e. Teacher assessment configuration.
11. Send the final prompt to the AI model through Groq API using LLaMA 3.3 70B.
12. Generate assessment content using the AI model.
13. Attach CLO labels beside each question or MCQ section in the generated assessment.
14. Format the generated assessment according to the selected assessment type.
15. Display the generated assessment and warning message, if any, to the teacher.
16. Allow the teacher to save, copy, preview, or download the generated assessment as a PDF.

End

6.2.2. Pseudocode of Gap Analysis Module:

6.2.2.1. Input:

- Question Paper
- Marksheet / Student Results
- Teacher Name
- Course Threshold

6.2.2.2. Output:

- Gap Analysis Report
- CLO Performance Summary
- Weak CLOs and Weak Students
- Question-wise Student Performance Heatmap
- Suggested Student Improvement Tasks

6.2.2.3. Begin

1. Receive question paper and marksheet uploaded by the teacher.
2. Verify the teacher session.
3. Match the question paper course code with the teacher's registered course records.
4. Load the threshold percentage set for the matched course.
5. Extract questions, marks, and CLO labels from the question paper.
6. Extract student roll numbers, names, marks, and question-wise scores from the marksheet.
7. If the question paper CLO labels are incomplete or different from the marksheet CLO mapping, reconcile the CLO mapping using the marksheet.
8. For each student in the marksheet:
 - a. Read the marks obtained in each question
 - b. Calculate performance against each CLO
 - c. Compare the score with the course threshold.
9. For each CLO:
 - a. Collect all questions mapped to that CLO
 - b. Calculate class performance for the CLO
 - c. Identify students who performed below the threshold
 - d. Mark the CLO as weak if performance is below the required threshold.
10. Generate question-wise student performance data for the heatmap.
11. Identify:
 - a. Weak CLOs
 - b. Weak students
 - c. Weak questions
12. Display the Gap Analysis Report to the teacher, including:
 - a. CLO performance summary
 - b. Weak CLOs
 - c. Weak students

- d. Question-wise student performance heatmap
 - e. CLO warning, if any
13. If the teacher requests improvement tasks:
 - a. Receive CIS file and difficulty level
 - b. Generate personalized questions for weak students
 - c. Allow teacher to send tasks to selected students.
 14. Save assigned student tasks and attachments in the system.
 15. Return the final gap analysis result to the teacher.

End

6.2.3. Pseudocode of Transformation Module:

6.2.3.1. Input:

- Source Document
- Template Document
- Output Type

6.2.3.2. Output:

- Transformed Document

6.2.3.3. Begin

1. Receive Source Document, Template Document, and Output Type from user
2. Identify the format of Source Document (PDF, Word, or Excel)
3. Extract structured data from Source Document using the appropriate Document Extractor
4. Extract column structure and layout from Template Document
5. Perform semantic mapping between:
 - a. Source document columns
 - b. Template document columns
6. Enrich column context using semantic information
7. Apply the template structure to the extracted source data
8. Generate transformed data aligned with the template format
9. Identify the required Output Type (PDF, Word, or Excel)
10. Export the transformed data using the appropriate Document Exporter

11. Provide the Transformed Document for download

End

6.3. Time Complexity

Time complexity is a method of expressing how the running time of an algorithm varies with the size of the input. It does not measure the actual time in seconds as the time it takes to execute the program may depend on many factors such as computer speed, programming language, processor, and memory. Rather, time complexity is concerned with the number of basic operations that an algorithm carries out for a growing input size.

We can understand the efficiency of an algorithm with the help of its time complexity. For instance, if an algorithm takes 10 inputs, 100 inputs, 1000 inputs, then time complexity provides an estimate of how many more times this algorithm will have to work at 10000 inputs. This is important because an algorithm that performs well on a small collection of data might be very slow if the data becomes large.

The time complexity of an algorithm is typically referred to as Big O notation, e.g. $O(1)$, $O(n)$, $O(n^2)$, $O(\log n)$. Big-O notation provides an overall hint at the rate of growth of an algorithm. It is useful for programmers to compare different algorithms and select the one that works better on large inputs. Hence, time complexity is a key concept while writing fast, efficient and scalable programs.

6.4. Retrieval-Augmented Generation

In Teacher Assist or Retrieval-Augmented Generation, the generated questions, assignments, exams, and content transformation are based on the uploaded course documents and are ensured using RAG. In addition to relying exclusively on the general knowledge of the AI model, the system first performs a search on the uploaded content and fetches the most relevant information based on the teacher's request. Then, the AI leverages this retrieved information to produce accurate, meaningful, and contextually relevant output.

This will increase the reliability of Teacher Assist, since the generated content remains aligned with the actual lecture notes, course files, CLOs, topics and academic content delivered by the teacher. For instance, if a teacher uploads lecture pdf to the system, and tells the system to generate quiz/exam questions, the system can use RAG to identify the topic of the pdf and generate the questions from it. Thus, RAG filters out random or irrelevant answers and enhances the precision, quality, and reliability of the created academic work.

6.5. Text Embedding

In Teacher Assist, the extracted text from uploaded documents is embedded in text, and then these text embeddings are converted to numerical vectors. Embeddings are a way to encode the meaning of words, sentences, and paragraphs in mathematical form, because computers are not directly able to understand human language as we do. Every text is turned into a vector, a set of numbers that represent the meaning of the text.

This helps to make it easier for the system to compare various portions of the document in terms of content rather than mere word-for-word. For instance, when the teacher requests questions about

“software testing,” the system will also return queries that contain “test cases”, “debugging” or “quality assurance,” although they might not be the same exact words.

In Teacher Assist, these embeddings are stored on a vector database, and then they can be later searched and retrieved when the RAG process comes into play. This enables the system to create more relevant, context-aware, accurate and recorded questions, assignments, exams and transformed content.

6.6. Vector Similarity Search Cosine Similarity

Teacher Assist uses vector similarity search to identify the most relevant sections of the document uploaded by the teacher in his/her generation request. Once this document text is extracted and converted into embeddings, the document text chunks are stored as numerical vectors in the vector database. When the teacher asks to create questions, assignments, exams or reworked content, this request is also transformed to a vector.

The system then searches for the closest match document vectors in the stored document vectors list that matches a request vector. Typically, this comparison is performed by cosine similarity: a measure of similarity based on the direction of two vectors. Two vectors with similar direction indicate that their meanings are similar.

In Teacher Assist, it assists a system in finding the most pertinent chunks of a document rather than the entire document or random content. For instance, if the teacher sets a question on a topic, the similar vector search can be used to find the precise portion related to the topic. The retrieved chunks are then fed into the RAG pipeline, which enables the system to produce accurate academic content that is document-based and context-aware.

6.7. Large Language Model Generation

The Large Language Model is what Teacher Assist uses to create quizzes, assignments, mid-term papers, final exam questions, and other academic content. The LLM then takes the retrieved relevant information from the uploaded documents as context and uses it to generate meaningful and structured output.

The LLM doesn't ask "what do you know? questions," rather it is based on the actual assignment material the teacher provides. This minimizes the chances that the questions produced are not tied to the uploaded lecture notes and course topics, CLOs and requested academic level. If the teacher asks for multiple-choice questions or descriptive questions on a given topic, for instance, the LLM can take the content from the retrieved documents and produce questions of the same type, on the same topic.

In Teacher Assist, the LLM also works to enhance the output by structuring the questions clearly, preserving the appropriate wording and adhering to the teacher's chosen structure, e.g. question type, marks, difficulty. This allows for more accurate, relevant and meaningful academic assessment of generated exams and assignments.

CHAPTER 7

7. Implementation

7.1. System Overview

Our teacher assists is an AI platform built entirely for our respected teachers to automate the generation of prompts according to their selection along with the student performance analysis. The system is developed by using a full stack architecture where postgresql is used for databases, angular is used for the frontend of the project and python (FASTAPI) is used for backend.

7.2. Code overview

7.2.1. Transformation Module Main.py

```
def finalize_transcript_artifacts(
    transcript_id: int,
    teacher_name: str,
    teacher_email: str,
    department_name: str,
    section: str,
    total_marks: float,
    selected_options: dict[str, str],
    assessment_columns: list[str],
    excel_rows: list[dict],
) -> None:
    db = SessionLocal()
    try:
        transcript = db.query(models.Transcript).filter(models.Transcript.id == transcript_id).first()
        if not transcript:
            return
        try:
            file_path = build_transcript_excel(
                sheet_id=transcript_id,
                teacher_name=teacher_name,
                teacher_email=teacher_email,
                department=department_name,
                section=section,
                total_marks=total_marks,
                selected_options=selected_options,
                assessment_columns=assessment_columns,
                students=excel_rows,
            )
            transcript.excel_file_path = file_path
            db.commit()
        except Exception as exc:
            print("Excel generation error:", exc)
            transcript.excel_file_path = ""
            db.commit()
    finally:
```

```

        db.close()
def build_transcript_detail(transcript: models.Transcript) -> schemas.TranscriptDetail:
    grouped_rows: dict[int, schemas.TranscriptRowDetail] = {}
    for mark in transcript.marks:
        if mark.student_id not in grouped_rows:
            grouped_rows[mark.student_id] = schemas.TranscriptRowDetail(
                student_id=mark.student_id,
                obtained_marks_by_assessment={},
                remarks=mark.remarks,
            )
        grouped_rows[mark.student_id].obtained_marks_by_assessment[mark.assessment_label] =
float(mark.obtained_marks)
        if mark.remarks:
            grouped_rows[mark.student_id].remarks = mark.remarks
    return schemas.TranscriptDetail(
        id=transcript.id,
        teacher_name=transcript.teacher.name if transcript.teacher else "",
        email=transcript.teacher.email if transcript.teacher else "",
        department_id=transcript.department_id,
        department=transcript.department.name if transcript.department else None,
        batch=transcript.batch,
        section=transcript.section,
        course_id=transcript.course_id,
        course=transcript.course.course_name if transcript.course else None,
        session_id=transcript.session_id,
        session=transcript.session.name if transcript.session else None,
        exam_type=transcript.exam_type,
        total_marks=float(transcript.total_marks),
        selected_options=transcript.selected_options,
        student_marks=list(grouped_rows.values()),
        created_at=transcript.created_at,
        download_url=f"/transcripts/{transcript.id}/download",
    )
@app.get("/health")
def health_check():
    return {"status": "ok"}
@app.get("/dropdown-options", response_model=list[schemas.DropdownOptionResponse])
def get_options(db: Session = Depends(get_db)):
    return db.query(models.DropdownOption).order_by(
        models.DropdownOption.category,
        models.DropdownOption.value,
    ).all()

```

7.2.2. Generation Module App.py

```

def generate_exam_route():
    data = request.json

```

```

session_id = data.get('session_id')
if not session_id or session_id not in sessions:
    return jsonify({'error': 'Invalid session. Please upload a file first.'}), 400
session = sessions[session_id]
if 'prompt' not in session:
    return jsonify({'error': 'Please generate a prompt first.'}), 400
try:
    # Parse the uploaded file — returns (text, images)
    text_content, images = parse_file(session['file_path'], session['file_type'])
    if not text_content or len(text_content.strip()) < 50:
        return jsonify({'error': 'Could not extract sufficient content from the file. Please ensure the
file has readable text.'}), 400
    # Analyze embedded images with llava (if any exist)
    image_descriptions = ""
    if images:
        print(f"[Image Analysis] Found {len(images)} image(s) in document — analyzing with
llava...")
        image_descriptions = analyze_all_images(images)
        if image_descriptions:
            print(f"[Image Analysis] Done. Descriptions injected into document content.")
    # Combine text + image descriptions into one content block
    combined_content = text_content
    if image_descriptions:
        combined_content += (
            "\n\n=== DOCUMENT IMAGES (Vision-Analyzed for Question Generation) ==="
            + image_descriptions
            + "\n\n=== END OF IMAGE ANALYSIS ==="
        )
    # Generate exam using AI
    exam_content = generate_exam(
        document_content=combined_content,
        prompt=session['prompt'],
        exam_type=session['exam_type'],
        mcq_count=session.get('mcq_count', 0),
        mcq_marks=session.get('mcq_marks', 1),
        theory_questions=session.get('theory_questions', [])
    )
    session['exam_content'] = exam_content
    return jsonify({
        'exam_content': exam_content,
        'images_analyzed': len(images),
        'llava_used': bool(image_descriptions and is_llava_available())
    })
except Exception as e:
    traceback.print_exc()
    return jsonify({'error': f'Failed to generate exam: {str(e)}'}), 500

```

7.2.3. Gap Analysis Module Main.py

```
@app.post("/api/student/tasks/assign")
async def assign_student_task(
    teacher_name: str = Form(...),
    student_roll_no: str = Form(...),
    title: str = Form(...),
    description: str = Form(""),
    question_content: str = Form(""),
    clo: str = Form(""),
    source_module: str = Form("gap_analysis"),
    course_code: str = Form(""),
    course_name: str = Form(""),
    reference_file: UploadFile | None = File(None),
    db: Session = Depends(get_db)
):
    teacher_name = normalize_text(teacher_name)
    student_roll_no = normalize_text(student_roll_no, uppercase=True)
    title = normalize_text(title)
    description = normalize_text(description)
    clo = normalize_text(clo, uppercase=True)
    source_module = normalize_text(source_module) or "gap_analysis"
    course_code = normalize_text(course_code, uppercase=True)
    course_name = normalize_text(course_name)
    if not all([teacher_name, student_roll_no, title]):
        raise HTTPException(status_code=400, detail="Missing task assignment fields")
    teacher = db.query(models.Teacher).filter(
        models.Teacher.teacher_name == teacher_name
    ).first()
    if not teacher:
        raise HTTPException(status_code=404, detail="Teacher not found")
    student = db.query(models.Student).filter(
        models.Student.roll_no == student_roll_no
    ).first()
    if not student:
        raise HTTPException(status_code=404, detail="Student with this roll number was not found")
    course = None
    if course_code:
        course = db.query(models.Course).filter(
            models.Course.course_code == course_code
        ).first()
    task = models.StudentTask(
        student_id=student.id,
        teacher_id=teacher.id,
        course_id=course.id if course else None,
        course_name_snapshot=course_name or (course.course_name if course else None),
```

```

    title=title,
    description=description or None,
    question_content=question_content or None,
    clo=clo or None,
    assigned_roll_no=student.roll_no or student_roll_no,
    source_module=source_module,
    status="assigned"

if reference_file and reference_file.filename:
    extension = Path(reference_file.filename).suffix
    saved_name = f"{uuid4().hex}{extension}"
    target_path = TASK_SUBMISSION_DIR / saved_name
    file_bytes = await reference_file.read()
    target_path.write_bytes(file_bytes)
    task.task_attachment_name = reference_file.filename
    task.task_attachment_path = str(target_path)
db.add(task)
db.commit()
db.refresh(task)
return {
    "status": "assigned",
    "task": build_student_task_payload(task, teacher),
    "student_name": student.student_name,
    "student_roll_no": student.roll_no or student_roll_no
}

```

7.2.3.1. Analysisi.py

```

router = APIRouter(
    prefix="/gap-analysis",
    tags=["Gap Analysis"]
)
@router.post("/")
async def gap_analysis(
    question_paper: UploadFile = File(...),
    transcript: UploadFile = File(...)
):
    questions = await parse_question_paper(question_paper)
    students = await parse_transcript(transcript)
    return analyze_gaps(questions, students, threshold_percentage=30)
@router.post("/with-recommendations")
async def gap_analysis_with_recommendations(
    question_paper: UploadFile = File(...),
    transcript: UploadFile = File(...),
    cis_file: Optional[UploadFile] = File(None),
    difficulty_level: str = Form("Moderate")
):

```



```

questions = await parse_question_paper(question_paper)
students = await parse_transcript(transcript)
gap_result = analyze_gaps(questions, students, threshold_percentage=30)
cis_weeks = []
clo_taxonomy = {}
course_title = ""
course_title = ""
course_title = ""
course_title = ""
if cis_file and cis_file.filename:
    try:
        # parse_cis_full se BOTH weeks aur taxonomy lo
        cis_data = await parse_cis_full(cis_file)
        cis_weeks = cis_data["weeks"]
        clo_taxonomy = cis_data["clo_taxonomy"]
        course_title = cis_data.get("course_title", "")
        course_title = cis_data.get("course_title", "")
        course_title = cis_data.get("course_title", "")
        course_title = cis_data.get("course_title", "")
        course_title = cis_data.get("course_title", "")
        course_title = cis_data.get("course_title", "")
        print(f"✓ CIS loaded: {len(cis_weeks)} weeks, taxonomy: {clo_taxonomy}")
    except Exception as e:
        print(f"⚠ CIS parse failed: {e}")
student_weak_clos: dict[str, list[str]] = {}
for clo_item in gap_result.get("clo_results", []):
    if clo_item["status"] == "Weak CLO":
        clo_label = clo_item["clo"]
        for name in clo_item.get("student_names", []):
            student_weak_clos.setdefault(name, []).append(clo_label)
weak_students = []
for student in students:
    name = student["name"]
    weak_clos = student_weak_clos.get(name, [])
    weak_students.append({
        "student_name": name,
        "roll_no": student.get("roll_no", ""),
        "weak_clos": weak_clos,
        "has_weakness": len(weak_clos) > 0
    })

return JsonResponse(content={
    "threshold_percentage": gap_result["threshold_percentage"],
    "summary": gap_result["class_summary"],
    "clo_overview": gap_result["clo_results"],
    "cis_loaded": len(cis_weeks) > 0,
})

```

```

        "course_title":    course_title,
        "clo_taxonomy":    clo_taxonomy, # ← frontend ko bhi de do
        "weak_students":    weak_students,
    })
    @router.post("/generate-for-student")
    async def generate_for_student(
        question_paper: UploadFile = File(...),
        transcript:    UploadFile = File(...),
        cis_file:    Optional[UploadFile] = File(None),
        student_name:    str = Form(...),
        weak_clos:    str = Form(...),
        clo_question_map: str = Form(...),
        difficulty_level: str = Form("Moderate")
    ):
        import json as jsonlib
        questions = await parse_question_paper(question_paper)
        cis_weeks = []
        clo_taxonomy = {}
        course_title = ""
        if cis_file and cis_file.filename:
            try:
                cis_data = await parse_cis_full(cis_file)
                cis_weeks = cis_data["weeks"]
                clo_taxonomy = cis_data["clo_taxonomy"]
                print(f"✓ CIS taxonomy for generation: {clo_taxonomy}")
            except Exception as e:
                print(f"⚠ CIS parse failed: {e}")
        weak_clos_list = jsonlib.loads(weak_clos)
        clo_q_map = jsonlib.loads(clo_question_map)
        generated = await generate_personalized_questions(
            student_name = student_name,
            weak_clos = weak_clos_list,
            all_questions = questions,
            cis_weeks = cis_weeks,
            clo_taxonomy = clo_taxonomy, # ← CIS se parsed taxonomy pass ho rahi hai
            clo_question_map = clo_q_map,
            difficulty_level = difficulty_level
        )
    return JSONResponse(content={
        "student_name": student_name,
        "weak_clos": weak_clos_list,
        "assignment": generated,
        "course_title": course_title})

```

7.3. Component Diagram

[Figure 7.1](#) shows how UI interacts with backend services that use AI/RAG modules and databases to process data, perform analysis, and generate intelligent outputs.

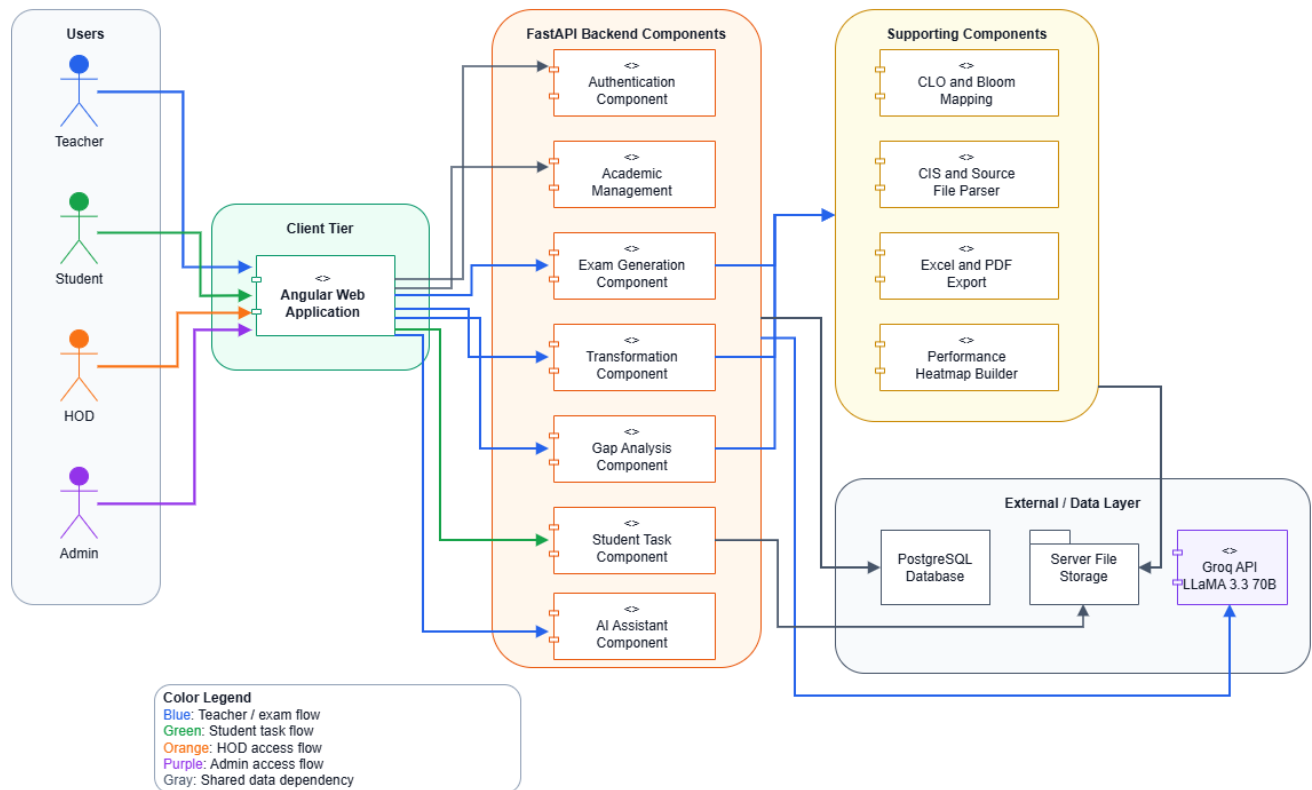


Figure 7.1: Component Diagram for Teacher Assist

7.4. Deployment Diagram

[Figure 7.2](#) illustrates the architecture where a web client connects to a Fast API backend that processes data, performs gap analysis and generation tasks, and integrates with AI, LLM APIs and vector databases for intelligent outputs.

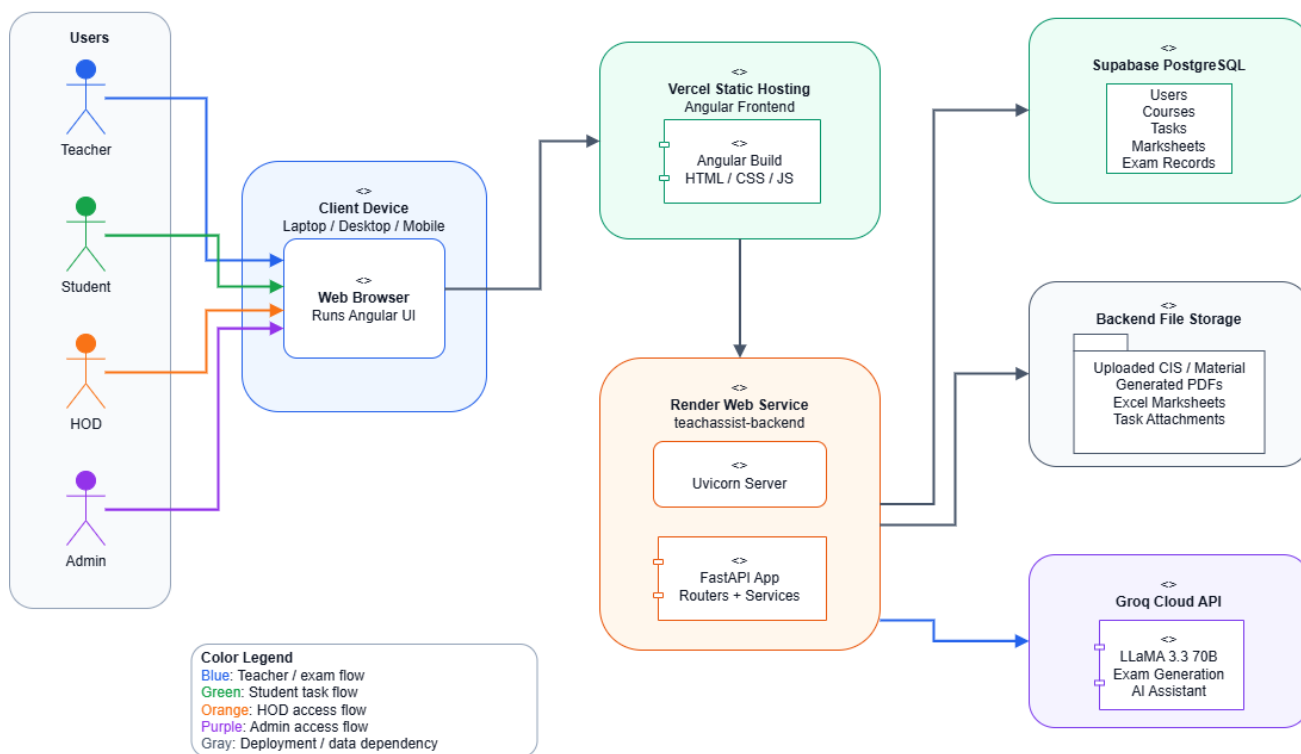


Figure 7.2: Deployment Diagram for Teacher Assist

CHAPTER 8

8. Testing

8.1. Introduction

To put the Teacher Assist system through its paces we tested it for teachers to create assessments and mark sheets, to identify learning gaps, to view student results for individual questions, and to provide additional support to underperforming students. The key aim of our testing was to ensure that all the system's various functions were working correctly to deliver the intended results.

To test the system, we conducted two types of tests: white box testing and black box testing. White box testing was used to test the system from the inside out. We tested functions that map learning objectives, tested the correct level of thinking, calculated grades for students, detected learning gaps for students and created heatmaps for students. The black box testing was conducted from the user's point of view. We tested the system by uploading different types of files, saving different types of grades, creating different types of assessments and testing the system with different inputs. We tested the system thoroughly to find bugs and to make sure that the system functions as intended.

8.2. Testing Plan

During testing phase, we focused on verification of system that performs all its functions correctly, correctly handles different types of inputs, and produces accurate results as expected.

8.2.1. Testing Scope

We tested each part of the Teacher Assist system to make sure everything was being tested. These include:

- Login system
- Uploading question papers (PDF/DOCX)
- Uploading transcripts (Excel/CSV)
- Running gap analysis
- Uploading CIS files
- Identifying weak students
- Generating content for each student
- Creating CSV files using the transformation module

8.2.2. Testing Types Used

We used a number of testing methods to properly assess the system. We applied black box testing, that is we tried to operate the system as an ordinary user by providing input and verifying the output. We also explored important components such as the gap analysis and the threshold calculations by using white box testing.

8.2.3. Tools Used for Testing

We used a few tools during testing to make the process easier and more effective:

- **Postman** to test backend APIs
- **pgAdmin** to check and verify database data
- Browser developer tools to debug frontend issues
- Manual testing to see how the system behaves from a user's point of view

8.3. Black Box Testing

8.3.1. Login Module Testing

In [Table 8.1](#) it ensures that only users with proper user credentials can log in the system has been tested using the login module.

Table 8.2: Login Module Test Cases

Test Case ID	Input	Expected Output	Actual Output	Result
BB-01	Correct ID & Password	Login successful	Login successful	Pass
BB-02	Correct ID, wrong password	Error message	Error message shown	Fail
BB-03	Wrong ID, correct password	Login fail	Login fail	Fail

8.3.2. File Upload

In [Table 8.2](#) The file upload module was evaluated to ensure only the required academic files that are valid files for processing can be uploaded.

Table 8.3: File Upload Test Cases

Test Case ID	Input	Expected Output	Actual Output	Result
BB-04	Valid PDF file	File uploaded	File uploaded	Pass
BB-05	Invalid file type (.jpg)	File rejected	File accepted	Fail
BB-06	Empty file	Error message	File uploaded	Fail

8.3.3. Weak Students + Generation + Transformation

In [Table 8.3](#) the module was tested to verify whether the system correctly identifies weak students after analysis and supports generation and transformation features.

Table 8.3: Weak Students and Generation and Transformation Test Cases

Test Case ID	Input	Expected Output	Actual Output	Result
--------------	-------	-----------------	---------------	--------

BB-19	Load weak students after analysis	Weak students shown	Correct data shown	Pass
BB-20	Load before analysis	Warning message	No message	Fail
BB-21	Generate for student	Content generated	Wrong student data	Fail
BB-22	API failure	Error handled	System crashes	Fail
BB-23	Empty form submits	Validation error	Form submitted	Fail
BB-24	Valid transformation input	CSV generated	Incorrect CSV	Fail

8.4. White Box Testing

White box testing is a software testing method where the internal structure, logic, and code of an application are tested to ensure proper operation. Conducted primarily by developers, it involves testing code paths, loops, and branches for security gaps.

8.4.1. Threshold Calculation Logic

In [Table 8.4](#) the threshold calculation logic was tested to check whether the system correctly identifies weak students based on marks percentage.

Table 8.4: Threshold Calculation Logic Test Cases

Test Case ID	Logic Under Test	Test Input	Expected Result	Status
WB-01	marks < 30%	20/100	Student should be weak	Pass
WB-02	marks = 30%	30/100	Boundary should be handled correctly	Fail
WB-03	negative marks	-5/100	Invalid input should be rejected	Fail
WB-04	marks > total	120/100	Invalid input should be rejected	Fail

WB-05	total marks = 0	10/0	System should stop calculation safely	Fail
-------	-----------------	------	---------------------------------------	------

8.4.2. CLO Mapping Logic

In [Table 8.5](#) the CLO mapping logic was tested to verify whether each question is correctly linked with its relevant Course Learning Outcome.

Table 8.5: Clo Mapping Logic Test Cases

Test Case ID	Logic Under Test	Test Input	Expected Result	Status
WB-07	one weak question in CLO	Q1 weak, Q2 strong	CLO should be weak	Pass
WB-08	missing CLO mapping	Question without CLO	Warning/error should appear	Fail
WB-09	wrong CLO mapping	Question linked to wrong CLO	Correct mapping should be ensured	Fail

8.4.3. Student Processing Loop

In [Table 8.6](#) the student processing loop was tested to check whether the system correctly processes uploaded files before analysis.

Table 8.6: Student Processing Loop Test Cases

Test Case ID	Logic Under Test	Test Input	Expected Result	Status
WB-19	valid PDF	paper.pdf	File should be accepted	Pass
WB-20	unsupported type	image.jpg	File should be rejected	Fail
WB-21	missing extension	file	File should be rejected	Fail
WB-22	uppercase extension	PAPER.PDF	File should be accepted properly	Fail

8.4.4. File Validation Logic

In [Table 8.7](#) the file validation logic was tested to check whether the system correctly filters student data based on weak and strong performance status.

Table 8.7: Validation Logic Test Cases

Test Case ID	Logic Under Test	Test Input	Expected Result	Status
WB-37	mixed weak and strong data	2 weak, 3 strong	Only weak students should appear	Pass
WB-38	no weakness status	Some students missing weak flag	System should handle safely	Fail
WB-39	all strong but weak shown	Strong-only dataset	Weak list should stay empty	Fail
WB-40	all weak but some missing	All below threshold	All weak students should appear	Fail
WB-41	repeated filtering	Filter run multiple times	Same result should appear every time	Fail

CHAPTER 9

9. Achievements

In this project we had the privilege of attending an Outcome Based Education workshop with the supervision of Sir Ghazanfar. The workshop was very informative and gave us insight on how to design learning outcomes, how to relate CLOs to assessments and how to implement evaluation strategies based on OBE principles. This experience helped us to understand the academic standards and to directly shape the development of the Teacher Assist system, particularly the gap analysis and assessment generation based on the CLOs.

We are also thankful to Sir Lubaid for his continued guidance and support during the project. His guidance has enabled us to appreciate how to practically implement the concepts of OBE and how this has influenced the direction of the system. He has brought a lot of feedback, encouragement and good ideas that have helped to enhance the quality of Teacher Assist and to enhance the clarity of our concepts.

CHAPTER 10

10. Discussion

This chapter gives a detailed discussion related to the overall development, working, testing and importance of the Teacher Assist system. The primary aim of this chapter is to explain the development of projects from conception to actual working system. It also provides some insights into the methods adopted in the project, the outcomes during testing, the significance of the various modules and the constraints encountered during the development of the project. Teacher Assist was created to ease teachers' academic workload by using AI technology to create assessments, analyze student performance, convert documents, and more.

The concept of Teacher Assist came from the observation of problems faced by teachers at academic institutions. It takes the teachers a lot of time to manually create quizzes, assignments, mid-term papers and final exam papers in traditional teaching and assessment. In addition, they must also make sure that the questions are appropriately mapped to Course Learning Outcomes and Bloom's Taxonomy levels. This can be tedious and complex, particularly if the class content is extensive, or if teachers have more than one type of assessment to develop. On a similar note, manually assessing student transcripts and determining which CLOs are weak can be very time-consuming and could result in some human error. The aim of the Teacher Assist was to overcome these difficulties by introducing an automated system for question generation, gap analysis, document processing and report generation.

A client-server system was used to implement the Teacher Assist system. This architecture means that the client side will communicate with the user, and the server side will do the bulk of the work. The client side offers a user-friendly interface for teachers to upload academic documents, choose the form of the assessment, enter the necessary details of the question, conduct a gap analysis, convert documents and download the reports. The server side performs the internal logic of the system such as reading the documents, extracting the text, retrieving the content, generating the text with AI, managing the DB, and formatting the output. Separate clients and server help to improve the overall organization of the project and makes the system easier to manage, test and update.

One of the key components of Teacher Assist is its use of Retrieval-Augmented Generation. To enhance the correctness and usefulness of the output, RAG is used. The system first looks for relevant content in the uploaded academic documents before generating questions based on that content generated by general AI knowledge. The system extracts the text from the document, and breaks it down into smaller meaningful parts, when a teacher uploads a document. These are transformed into vector representations and saved as vectors in a vector database. For the question generation request, the system will use the content it has stored and provide the most relevant content based on the user's request. The content retrieved is then fed into the AI model, which creates questions from the actual course material.

Having RAG increases the reliability of the system since the questions produced are tied to the uploaded document. This decreases the likelihood of creating non-specific or generic questions. For instance, a teacher uploads a lecture document related to software design, the system fetches relevant content from the document and creates questions based on the same topic. This can help guarantee that the content is relevant, content area specific, and of educational value. RAG can also facilitate

the system to process larger documents by focusing only on the most pertinent parts instead of the whole document.

Teacher Assist's question generation module is one of the crucial modules. This module enables teachers to create various assessments, including quizzes, assignments, mid-term and final exam papers. The teacher can set the number of questions, how the points will be distributed, the type of questions, the CLOs, and the Bloom's taxonomy levels. This provides greater structure to the assessment created and more closely mirrors the principles of Outcome-Based Education. System can be created to produce MCQs, short questions, long questions and case-based questions as per the requirements. This will save teachers time and allow consistency in assessment preparation as well.

CLO and Bloom's Taxonomy alignment is another important aspect of the system. For outcome-based education, question maps are important with respect to learning outcomes and cognitive levels. To help with this requirement, Teacher Assist enables teachers to create questions based on the selected CLOs and Bloom level. For instance, if a teacher creates a set of questions for each of the four Levels of Analysis, that might include some for “Remember” at the basic concept level, some for “Understand” at the explanation level, some for “Apply” at the practical level, and some for “Analyze” at the comparison and reasoning level. This will make the assessment more balanced and make it possible to assess students at various levels of learning.

Teacher Assist also has a major contribution to make in the form of gap analysis module. This module is designed for teachers to be able to determine weak students and weak CLOs from the transcript. The teacher uploads or enters the student marks, and associates’ questions with CLOs in this module. The system then compares the marks it obtained with a given threshold value. If a student doesn't reach the minimum score requirements in the questions associated with a CLO, that CLO is considered weak for the student. This assists teachers to get an idea of what learning outcomes they are not doing well, and which students need to be improved. Gap analysis results can be presented in tables, charts and reports for ease of teacher interpretation.

Teacher Assist was designed in a modular way, with various modules developed separately, to allow for the modularity and maintenance of the project. The primary modules comprise of authentication, document upload, text extraction, embedding generation, retrieval, question generation, gap analysis, transformation, database handling, and report generation. Every module has functionality and can communicate with other modules as necessary. Easy to debug and improve, thanks to this modular approach. The system does not have to be changed if one module requires updating. For instance, if the AI model needs to be replaced or enhanced, the entire front-end interface does not need to be re-designed.

Testing was also very important as part of the project as it enabled the system to be tested to ensure that it was running as specified. Key features of Teacher Assist were tested using a variety of test cases. The system has been tested for login, upload documents, text extraction, question generation, gap analysis, transformation, and download the report. These tests were conducted to check whether the correct input is being provided, the data is being processed correctly, and the output is being generated accordingly. Testing also uncovered bugs and the reliability of the system was enhanced.

In the evaluation of the system, both black-box testing and white-box testing were taken into consideration. The system was tested from user's point of view using black box testing. This form of testing involved testing different inputs to the system and testing the outputs but did not pay attention to the code inside the system. A teacher uploads a document and chooses Quiz Generation; it is expected that the system generates Quiz questions from the document. A white-box testing method was applied to verify the inside logic of the system. This was done by testing the threshold calculation in gap analysis, the weakness detection in CLO, retrieval process and the conditions in the modules. These tests enabled the external behavior and internal logic of the system to be ensured to be correct.

The outcomes of the tests carried out indicated that the main functions of Teacher Assist can be done successfully. It could upload documents, extract content and retrieve relevant information, generate questions, complete gap analysis and create a report. The RAG-based retrieval process ensured that the questions generated were primarily relevant to the uploaded content, as evidenced by the most relevant content. The gap analysis module could identify weak students and/or weak CLOs that were determined by the threshold. The results indicate that the system satisfies the major functional needs and that the system offers some support for the teachers.

In developing and testing the system, however, some limitations were also noted. A problem is that the quality of the questions is dependent upon the quality of the document that is uploaded. The extracted text may not be accurate if the document is incomplete, unclear, or is a scan or poorly formatted, which can impact the output. One challenge is that large documents can take longer to process as they need to be extracted, chunked, embedded, retrieved, and have an AI response generated. AI-generated content should also be reviewed by a teacher, as the AI is meant to support teachers and not to replace teacher judgment.

The other challenge is with regard to the performance of AI models and reaction time. The time it takes may vary if the AI model is running locally, depending on the system's hardware capabilities and the size of the prompt. If an external API is used, the performance can be greater, but it can rely on the internet speed and availability, API limits and API response time. All these factors can impact on the overall performance of the system. Therefore, future improvements can focus on optimizing document processing, improving retrieval accuracy, and using faster AI models.

The project also has academic significance as it is based on the principles of Outcome Based Education. Having content as the focus is not only on OBE but also measuring the attainment of the necessary learning outcomes. Teacher Assist facilitates this through the generation of questions based on CLOs as well as Bloom's Taxonomy and analyzing student performance in relation to CLOs. This would help the system for teachers, departments and institutions who are practicing OBE-based assessment.

That said, the system is not without its drawbacks. Whatever you upload, you will get the output of the same quality. When the document is poorly structured, not complete, or a scanned image, or the document's text cannot be understood perfectly, the result returned may contain noisy data. It can impact both the retrieval process and generated final questions. Likewise, if the document is more

substantial in size, the system might require less time to process the document, as the system needs to extract text, chunk it, generate embeddings and retrieve the content in relevance context. Another drawback is the basic lack of dependence on AI, as the system's execution depends on the selection of the AI model, framework assets, web.

Despite these constraints, Teacher Assist has great potential to be improved in the future. For future versions, CLO mapping and mapping of content to CLO can be automated, so that the system can identify automatically which content belongs to which CLO. Advanced analytics can also be added to display more detailed charts, student progress trends and CLO achievement percentages. It can be created a student portal where students can see their weaknesses and activities they should engage in to improve. The system can also integrate with LMS for integration with existing university learning systems. Question generation and analysis can also be further enhanced in quality, speed and accuracy using improved AI models.

Thus, Teacher Assist may be a helpful, expandable academic assistance system. It is used to create and automate all assessments, assist in the implementation of Outcome Based Education, assist in student learning gaps identification, and enhance academic documentation. The system is designed in a manner that is extendable and further improvement will be possible for future use based on institutional needs. With its AI capabilities, RAG, client-server design, and academic modules, it is a robust educational solution for today's classrooms.

CHAPTER 11

11. Future Work

Teacher Assist delivers its key aims and academic support activities well, but there are many opportunities for future enhancement and development. With the advancement of technology and the requirements of education, more functions can be added into the system, which will be more intelligent, efficient, easy to use and appropriate for large academic environments. In the future, additional advancements have the potential to enhance the quality of the outputs that can be generated, increase automation, reinforce academic analysis, and increase teacher and student experience.

The next big improvement for Teacher Assist is that the automatic CLO-mapping will be replaced. Currently, teachers need to manually link the generated questions to Course Learning Outcomes. Natural Language Processing and AI semantic analysis could be utilized to automatically find the most relevant CLO to a generated question or uploaded course topic in future versions of the course. This feature will save teachers the time and effort required to align CLOs and increase the consistency and accuracy of CLO alignment. Automatic CLO mapping will also benefit institutions following the standards of Outcome Based Education as it will structure and make assessment preparation more efficient.

A further enhancement is the inclusion of a sophisticated taxonomy of Bloom's. The system now allows for the generation of questions based on selected Bloom levels, but the future versions can be extended to intelligent classification whereby the system will automatically detect the cognitive level of a question generated. The system will be able to recognize at the level at which the question is located – Remember, Understand, Apply, Analyze, Evaluate, create – without having to select manually. This option may be used to help the teacher to create a more well-rounded and subject-appropriate evaluation.

The integration of Learning Management Systems is also an important future possibility. Teacher Assist can integrate with platforms like Moodle, Blackboard, Canvas or Google Classroom. LMS will allow teachers to upload course content directly from the institutional system, create assessments, publish online quizzes, keep student records, and monitor student performance from one place. This integration will enhance workflow efficiency and make the system more user-friendly for universities and educational institutions.

One of the other significant improvements to come soon is the creation of a complete student portal. Teacher Assist currently primarily supports teachers, but in future versions students can have dedicated access to Teacher Assist too. Students can check their marks, weak CLOs, performance report, generate feedback and recommend improvement material via the student portal. Pupils are given test questions tailored to their weaker aspects too. This will enhance the interactivity of the system and will bring better improvements of the learning outcomes of the students.

Advanced analytics and visualization dashboards can also be incorporated into the system to improve its capabilities. The system is currently primarily a textual reporting system, but future versions will also feature interactive graphs, charts, performance comparisons and visual summaries. The teachers and administrators can see class performance trends, percentages for each CLO, weak

topic analysis, student progress over time and overall academic statistics. Visual knowledge will assist institutions in making better academic decisions and monitor educational performance more efficiently.

Multi-language content processing and generation is another future improvement. The system at present is mainly English academic content but future versions can be available in the Urdu language and other regional or international languages. This feature will help Teacher Assist to be more widely available in institutions where bilingual/multilingual teaching is practiced. Multi-language support can also enable teachers to create assessments and reports in their language of choice.

In future versions, the document processing module can be greatly enhanced. Currently, the system can display digital documents that can be read but could be extended to display scanned PDFs and images using Optical Character Recognition (OCR) technology. This will help the system to retrieve the text from the academic documents even if they are handwritten and/or scanned. Improved OCR support will increase versatility and ease of use of the entire system for educational purposes.

The other significant improvement is plagiarism and similarity checking features. The system can be enhanced in future releases by incorporating AI features for plagiarism detection to guarantee content originality and minimize plagiarism. A second application of similarity analysis is for teachers to verify that composition(s) produced and submitted by students is authentic and appropriate for academic use. It will contribute to the academic integrity and reliability.

The use of additional AI and LLM advancements for improving the capabilities of Teacher Assist is also possible. Potential future AI models can provide more accurate understanding of the academic content, improved reasoning, increased speed at generating responses, and improved accuracy of generating questions. Proceeding to improve the integration of AI can also generate concept-based questions instead of just factual questions. Moreover, performance will be boosted, and system response time will be shortened with optimized AI models during huge document processing.

Other possible enhancements might be with adaptive assessment generation. Here, the system can automatically vary the level of difficulty or skills of the students in the questions. For instance, if they perform well, they can be given more analytical or higher order questions and if they perform poorly, they can be given simpler conceptual questions. This adaptive system has the potential for tailoring to individual learning needs and improving assessment.

Teacher Assist may also come with features such as AI-based recommendations and predictive analysis in future releases. The system can analyze the student's performance trend and forecast students in risk of poor academic performance. Based on the analysis, the system can recommend the learning resources, revision resources, practice exercises and/or improvement strategies. This capability will be useful for teachers to recognize the need for earlier intervention for students who are struggling and offer academic intervention as soon as it is needed.

Voice interaction and intelligent support is another helpful addition. The teacher can give directions either verbally or via speech-to-text. Likewise, AI chat support can be incorporated to enable

teachers to swiftly produce questions, summaries, or reports via conversational communication. This will help make the system more user friendly and interactive.

Additionally, the system can be enhanced by adding real-time collaboration tools. All teachers/Department members can collaborate in the same platform to prepare the question papers, map the CLOs, review documents, and analyse assessments. Collaboration support will help to streamline interactions and minimize repetitive efforts within the faculty.

Security enhancements are also a crucial point that needs to be addressed in future research. The system can also provide role-based access control, two-factor authentication, encrypted storage and secure cloud backup to enhance data security. Academic records and student information are sensitive and robust security measures will ensure reliability and trust in it.

Another potential future development is cloud-based deployment and scalability. The system can presently be executed on a local or constrained server system but can be extended to cloud based platforms to increase accessibility, storage management, backup, scalability and system performance. The deployment of the clouds will also enable multiple institutions, departments, and teachers to use the system simultaneously from different locations.

The use of automatic report generation and accreditation support can also be added to Teacher Assist to improve it. For institutions based on OBE or accreditation systems, academic records and achievement reports for CLO may be necessary. Reports for accreditation, semester summaries, and analysis will be created automatically in future versions, saving administrators time as well.

Question quality evaluation mechanisms is another future enhancement. AI can assess the balance, meaningfulness, grammar and academic appropriateness of created questions. This aspect can help make the assessments which are created more reliable and can lessen manual editing.

In future releases, Teacher Assist may be synced up with mobile apps as well. The system can be implemented as mobile to enable teachers and students to use the system via their smartphones or tablets. Documents can be uploaded by teachers and performance reports and recommendations can be followed up by the teachers from anywhere, while student can follow-up on performance reports and recommendations more easily.

Teacher Assist could be a better platform for the future, being the integration of AI, Retrieval-Augmented Generation, document processing, assessment generation, and academic analysis into one. This system is helpful for teachers, and can be developed to become a full-fledged intelligent educational assistant that can help institutions on a larger scale.

Lastly, additional enhancements are planned for Teacher Assist to further improve its effectiveness and usability, such as AI-powered automatic CLO mapping, in-depth Bloom's taxonomy analysis, integration with LMS systems, advanced AI models, plagiarism detection, OCR support, cloud deployment, student portals, and multi-language support. These improvements will make the system more scalable, intelligent and flexible towards meeting the current learning requirements. The

system will continue to grow and be enhanced with new advanced technologies and, one day, Teacher Assist may become a one-stop smart academic management and assessment system for educational institutions.

CHAPTER 12

12. Conclusion

Teacher Assist is a great example of how Artificial Intelligence, Retrieval-Augmented Generation and the latest in software engineering can be integrated to enhance the academic assessment and educational management system. The primary goal of this work is to design and develop an intelligent academic support system that will enable teachers to generate quizzes and assignments from academic documents uploaded to the system, generate mid-term papers and final examination papers from academic documents, perform a gap analysis based on the CLO of the uploaded documents, transform the uploaded academic documents to other formats, and generates a report based on the uploaded documents. The system was developed keeping in mind the objective of minimizing manual work, improving efficiency and facilitating the use of Outcome Based Education practices in educational institutions throughout the entire development process.

Teacher Assist is an organized and automated answer to a lot of problems that teachers come across when preparing for academic assessment. The academic assessment process has traditionally been a time-consuming process for teachers, involving manual study of academic documents, designing questions, linking them to Course Learning Outcomes and Bloom's Taxonomy levels, reviewing student performance and drafting reports. remain tethered to the actual content given to students by their instructors This feature assists in making the system more academically accurate and usable for educational content.

The use of CLO based Gap Analysis is another key success of this project. This module could be useful to identify weak students and/or weak learning outcomes by comparing students' performance with threshold values and analysing students' transcript. Teacher's analysis of this will help identify student weaknesses and correct academic actions can be taken. This can help strengthen the concepts of Outcome Based Education and act as a positive element in helping to improve teaching and learning.

In addition, the system offers functionalities to transform documents and generate reports, which further increase the usefulness and comprehensiveness of the system as an academic assistance tool. This generates output which can be downloaded as a PDF or DOCX file to be readily used for academic documentation, print and institutional recordkeeping. There is a modular approach to the project, with everything having a defined role and being structured with the other components of the project.

Testing and evaluation of the major functionalities of the system were successfully carried out. The requirements were met by uploading documents, extracting text, retrieving the relevant information, creating questions, gap analysis, and creating reports. A variety of tests were carried out to ensure that the system is running correctly and providing meaningful results. There were some disadvantages experienced such as the time taken to process large documents and reliance on the quality of the documents, however, the project has been positive and satisfactory.

There's definite scope for further improvements to be made to Teach Assist. All the extra features such as automatic CLO mapping, LMS integration, advanced analytics, plagiarism check, OCR support, predictive analysis, cloud deployment and student portals can further augment the

functionality and scalability of the system. These future possibilities show that the project is not just intended for use as a question generator, but could be a complete intelligent learning management system.

Overall, Teacher Assist has been successful in meeting its goals and it shows how AI can be applied to the field of education. The system reduces human efforts, automates the generation of the test structure, expands academic analysis and provides a more intelligent solution for educational tasks. It unifies various technologies such as AI, RAG, vector databases, document processing, and client-server architecture in a single platform that can be seamlessly used by both teachers and educational institutions.

There's definite scope for further improvements to be made to Teach Assist. The integration of additional features like automatic CLO mapping, LMS integration, advanced analytics, plagiarism check, OCR support, predictive analysis, cloud deployment, and student portals can further enhance the functionality and scalability of the system. These future possibilities reveal that the project has no intention of just being a question generator but can become a full intelligent learning management system.

Overall, Teacher Assist has been successful in meeting its goals and it shows how AI can be applied to the field of education. The system minimizes human effort, facilitates the automation of test structure generation, enhances academic analysis and offers a more intelligent approach to educational tasks. It unifies various technologies such as AI, RAG, vector databases, document processing, and client-server architecture in a single platform that can be seamlessly used by both teachers and educational institutions.

Finally, the creation of Teacher Assist as a Final Year Project offered valuable learning, hands-on experience, and technical insights into the domains of Artificial Intelligence, software engineering, database management, document processing, and academic automation. The project enabled us to implement theoretical concepts to a practical situation in the education field and appreciate the significance of working as a team, designing a system, testing, and conducting research in the software development process. It was the best choice as our FYP since it was not only a real academic problem that was being addressed but it was also an opportunity to design a system that was both meaningful, innovative and also impactful to the education sector in a positive way. And also, we are proud enough to present Teacher Assist as our FYP and Sir Lubaid being our supervisor.

CHAPTER 13

13. Appendices

13.1. Functional Requirements:

APPENDIX-A

[Table 13.1](#) describes the main functions that the Teacher Assist system must perform. It provides details about the necessary qualities that are needed to assist teachers and academic work.

FR ID	Functional Requirements Description
FR-1	The system shall allow teachers to login using their username and password
FR-2	The system shall support uploading academic document (pdf, ppt, doc)
FR-3	The system should automatically extract text from the documents
FR-4	The system shall store embedding in a vector database for retrieval
FR-5	The system should generate quiz, assignments, mid exam as well as final exam from the uploaded documents
FR-6	The system should also perform transformation and gap analysis

Table 13.1: Functional requirements for Teacher Assist

13.2. Non-Functional Requirements:

APPENDIX-B

[Table 13.2](#) describes the quality criteria of the system. It is the efficiency, reliability and ease of operation of the system that is the focus

NFR ID	Category	Non-Functional Requirements Description
NFR-1	Usability	The system shall be easy to use for teachers
NFR-2	Performance	The system shall generate outputs within acceptable response time
NFR-3	Maintainability	The system shall be modular and easy to update
NFR-4	Portability	The system shall run on multiple operating environments
NFR-5	Reliability	The system shall provide consistence and stable performance

Table 13.2: Non-Functional requirements for Teacher Assist

13.3. Hardware requirements:

APPENDIX-C

[Table 13.3](#) gives the basic hardware requirements for the proper operation of the system. It guarantees sufficient processing capabilities, memory, and storage.

Component	Requirements
Processor	Intel Corei5 or higher
RAM	Minimum 8GB
Storage	Minimum 50 GB free space

Table 13.3: Hardware requirements for Teacher Assist

13.4. Software requirements:

APPENDIX-D

[Table 13.4](#) introduces software technologies for system development. This involves the tools required for frontend, backend, database and AI processing

Component	Requirement
Frontend	Angular Framework
Backend	Python with Fast API
Database	PostgreSQL
Vector Database	Chroma DB
AI Engine	Large Language Models (LLMs)

Table 13.4: Software requirements for Teacher Assist

13.5. Input and Output Requirements

APPENDIX-E

[Table 13.5](#) input data to each module and output from each module is explained. It can be used to demonstrate the flow of data through the system

Module	Input	Output
Question Generation	Uploaded document, exam type, question count, marks, Bloom level	Quiz, assignment, mid-term or final exam paper
Gap Analysis	Question paper, transcript, threshold value	Weak CLOs, weak students, analysis report
Document Transformation	Source file and required template	Transformed document
Document Extraction	PDF, DOCX, PPTX file	Extracted text content
Report Generation	Generated data or analysis result	Downloadable PDF/DOCX report

Table 13.5: Input and Output Requirement for Teacher Assist

13.6. Supported File Formats

APPENDIX-F

[Table 13.6](#) provides a list of the types of files that Teacher Assist can support. Displays the formats that can be uploaded and processed by the system.

File Type	Supported Format	Purpose
PDF	.pdf	Used for lecture notes, books, and academic documents
Word Document	.docx	Used for editable academic content
PowerPoint	.pptx	Used for lecture slides and presentation material
Excel	.xlsx	Used for transcripts and structured student data
CSV	.csv	Used for tabular data and transformation input

Table 13.6: Supported File Format for Teacher Assist

13.7. Module Description

APPENDIX-G

[Table 13.7](#) demonstrates the interdependence of various components of the system.

Module Name	Description
Login Module	Allows teacher access to the system in a safe manner
Document Upload Module	Supports users to upload pieces of academic text for processing
Text Extraction Module	Reads text from an uploaded PDF, DOCX, or PPTX file
Embedding Module	Extracts text from image and converts it to vectors
Retrieval Module	Gets the most similar document content
Question Generation Module	Stimulates academic questions as per chosen exam type and Bloom level
Gap Analysis Module	Identifies weak students and weak CLOs based on the transcript data

Table 13.7: Module Description for Teacher Assist

13.8. Software Requirements

APPENDIX-H

[Table 13.8](#) describes the tools used in the development and implementation of the system.

Software/Tool	Purpose
Angular	Front-end user interface development
Fast API	Back-end API development
PostgreSQL	Database management
Chroma DB / Vector Database	Storing document embeddings for retrieval
Python	Back-end logic and AI processing
Node.js	Running Angular development environment
PG Admin	Managing PostgreSQL database
VS Code	Code editing and project development
Ollama / Groq API	AI-based question generation

Table 13.8: Software Requirements for Teacher Assist

13.9. Bloom's Taxonomy Levels Used

APPENDIX-I

[Table 13.9](#) explained in this section are those used for creating questions. It assists in making questions generated in accordance to proper academic learning standards.

Bloom Level	Meaning	Example Question Keyword
Remember	Recall basic facts or concepts	Define, List, Identify
Understand	Explain ideas in own words	Explain, Describe, Summarize
Apply	Use knowledge in a practical situation	Apply, Solve, Demonstrate
Analyze	Break information into parts	Analyze, Compare, Differentiate
Evaluate	Making judgments based on criteria	Evaluation, Justify, Critique
Create	Produce new ideas or solutions	Design, Develop, Construct

Table 13.9: Blooms Taxonomy Levels Used in Teacher Assist

CHAPTER 14

14. References

- [1] Arafeh, S. (2015). Curriculum mapping in higher education: a case study and proposed content scope and sequence mapping tool. *Journal of Further and Higher Education*, 40(5), 585–611. <https://doi.org/10.1080/0309877X.2014.1000278>.
- [2] M. A. Chowdhury, K. K. S. Chisty, H. Tushar, K. M. F. Ahmed, and S. A. Waliullah, “Automating assessment and evaluation for a bachelor’s degree program,” *International Journal of Evaluation and Research in Education (IJERE)*, vol. 12, no. 4, pp. 2037– 2044, Dec. 2023, doi: 10.11591/ijere.v12i4.25479.
- [3] Shaheen, M. (2021). Decision tree for PLOs of undergraduate computing program based on CLO of computer programming. *Interactive Learning Environments*, 31(4), 2452–2470. <https://doi.org/10.1080/10494820.2021.1890621>.
- [4] N. Zaki, S. Turaev, K. Shuaib, A. Krishnan, and E. Mohamed, “Automating the mapping of s to using for accurate educational program evaluation,” *Education and Information Technologies*, vol. 28, pp. 16723–16742, May 2023, doi: 10.1007/s10639023-11877-4.
- [5] B. S. Bloom, M. D. Engelhart, E. J. Furst, W. H. Hill, and D. R. Krathwohl, *Taxonomy of Educational Objectives: The Classification of Educational Goals, Handbook I: Cognitive Domain*. New York, NY, USA: Longmans, Green, 1956.
- [6] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] S. Alahmari, E. Alqahtani, and M. Alshayeb, “Outcome-Based Education Implementation Challenges in Higher Education,” *IEEE Access*, vol. 9, pp. 136905–136917, 2021.
- [8] A. S. Drigas and M. A. Pappas, “Online and Other ICT-Based Assessment Tools for Problem-Solving Skills,” *International Journal of Emerging Technologies in Learning (iJET)*, vol. 10, no. 5, pp. 4–11, 2015.
- [9] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proc. NAACL-HLT*, 2019.
- [10] A. Holmes, M. Bialik, and C. Fadel, in *Education: Promises and Implications for Teaching and Learning*. Center for Curriculum Redesign, 2019.

- [11] M. M. Syeed, A. Shihavuddin, M. F. Uddin, M. Hasan and R. H. Khan, " (OBE): Defining the Process and Practice for Engineering Education," in IEEE Access, vol. 10, pp. 119170-119192, 2022, doi: 10.1109/ACCESS.2022.3219477.
- [12] Z. Zhao, C. Yang and Q. Wu, "TwoWin-SOVA: Two Windows Discrete Cosine Transform and Synthetic Minority Oversampling Technique One-Versus-All Ensemble Classifiers for Imbalanced Hyperspectral Image Explainable Classification," in IEEE Transactions on Geoscience and Remote Sensing, vol. 61, pp. 1-16, 2023, Art no. 5520216, doi: 10.1109/TGRS.2023.3307123.
- [13] D. Zhao, L. Cui and D. Liu, "Bearing Weak Fault Feature Extraction Under Time-Varying Speed Conditions Based on Frequency Matching Demodulation Transform," in IEEE/ASME Transactions on Mechatronics, vol. 28, no. 3, pp. 1627-1637, June 2023, doi: 10.1109/TMECH.2022.3215545.
- [14] William, I. O., & Altamimi, M. (2024). Text Embedding Implementation Using Retrieval Augmented Generation (RAG) Model Combined With Large Language Model. International Journal of Advanced Natural Sciences and Engineering Researches, 5(5).
- [15] William, I. O., & Altamimi, M. (2024). Text Embedding Implementation Using Retrieval Augmented Generation (RAG) Model Combined With Large Language Model. International Journal of Advanced Natural Sciences and Engineering Researches, 5(5).
- [16] Nam, D., Macvean, A., Hellendoorn, V., Vasilescu, B., & Myers, B. (2024, April). Using an llm to help with code understanding. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (pp. 1-13).
- [17] Saha, G. C., Akber, S. M., & Roy, A. (2023). Impact of Outcome-Based Education (OBE) on Learners' Performance in Business Courses. International Journal of Professional Business Review: Int. J. Prof. Bus. Rev., 8(8), 6.
- [18] Kobayashi, H., & Takagi, M. (2024, December). Learning by Quiz Creation: A Novel Approach Using Interactive Generative AI and Its Experimental Application. In 2024 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE) (pp. 1-8). IEEE.
- [19] Gabajiwala, E., Mehta, P., Singh, R., & Koshy, R. (2022, November). Quiz maker: Automatic quiz generation from text using NLP. In Futuristic trends in networks and computing technologies: Select Proceedings of fourth international Conference on FTNCT 2021 (pp. 523-533). Singapore: Springer Nature Singapore.
- [20] Dijkstra, R., Genç, Z., Kayal, S., & Kamps, J. (2022, July). Reading Comprehension Quiz Generation using Generative Pre-trained Transformers. In iTextbooks@ AIED (pp. 4-17).

- [21] Shepard, R. N. (2024). Form, formation, and transformation of internal representations. In *Information processing and cognition* (pp. 87-122). Routledge.
- [22] Shepard, R. N. (2024). Form, formation, and transformation of internal representations. In *Information processing and cognition* (pp. 87-122). Routledge.
- [23] Alordiah, C. O. (2023). Mind the gap: exploring effective strategies for conducting gap analysis in educational studies. *Journal of Contemporary Issues in Education*, 7(1), 1-14.
- [24] Okoye, K., Arrona-Palacios, A., Camacho-Zuñiga, C., Achem, J. A. G., Escamilla, J., & Hosseini, S. (2022). Towards teaching analytics: a contextual model for analysis of students' evaluation of teaching through text mining and machine learning classification. *Education and Information Technologies*, 27(3), 3891-3933.
- [25] Zayani, H. M., Abdelfattah, W., Sellami, R., Slimane, J. B., & Kachoukh, A. (2024). A Framework for Efficient and Accurate Automated CLO and PLO Assessment. *Engineering, Technology & Applied Science Research*, 14(2), 13362-13368.
- [26] P. Nerisafitra, Martini Dwi Endah Susanti, Ronggo Alit, and Aditya Prapanca, "How to Find Gap Analysis in Undergraduate User Satisfaction Through a Tracer Study", *ATCSJ*, vol. 6, no. 2, pp. 123–131, Sep. 2024.
- [27] Teferi GB, Gu Q, Wu Z. Outcome-Based Education (OBE) for the Integration of Mechanical and Electrical Engineering Teaching Method. *International journal of Technology in Education Science*. 2025 Jun 4;2(1):53-78.
- [28] Mahrishi, M. and Abbas, A., 2023, October. Assessment of knowledge gaps in outcome-based education (OBE) among engineering faculty members: an empirical study. In *2023 IEEE International Conference on Engineering Veracruz (ICEV)* (pp. 1-6). IEEE.
- [29] Zamir, M.Z., Abid, M.I., Fazal, M.R., Qazi, M.A.A.R. and Kamran, M., 2022. Switching to outcome-based education (OBE) system, a paradigm shift in engineering education. *IEEE Transactions on Education*, 65(4), pp.695-702.
- [30] Qadir, J., Shafi, A., Al-Fuqaha, A., Taha, A.E.M., Yau, K.L.A., Ponciano, J., Hussain, S., Imran, M.A., Muhammad, S.S., Rais, R.N.B. and Rashid, M., 2020, June. Outcome-based (engineering) education (OBE): International accreditation practices. In *2020 ASEE Virtual Annual Conference Content Access*.
- [31] Wang, T., Schwartz, D. and Lingard, R., 2008. Assessing student learning in software engineering. *Journal of Computing Sciences in Colleges*, 23(6), pp.239-248.

- [32] Venkatesh, K.A. and King, C.S., 2021. Challenges and Issues in Implementation of OBE. Assessment tools for mapping learning outcomes with learning objectives, pp.83-96.
- [33] Dong, Q. and Liu, X., 2023. Optimization practice of university innovation and entrepreneurship education based on the perspective of OBE. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 118, pp.181-189.
- [34] Dong, Q. and Liu, X., 2023. Optimization practice of university innovation and entrepreneurship education based on the perspective of OBE. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 118, pp.181-189.
- [35] Chan, M.K., Wang, C.C. and Arbai, A.A.B., 2022. Development of dynamic OBE model to quantify student performance. *Computer applications in engineering education*, 30(5), pp.1293-1306.
- [36] Gamboa, R.A. and Namasivayam, S., 2013. A blueprint of software enabled quantitative measurement of programme outcomes: A case study for Taylor's University. *Journal of Engineering Science and Technology*, 8(Special Issue), pp.67-79.
- [37] Rahate, V., 2020. Impact of (OBE) on teaching effectiveness of faculty members of professional program. *Indian Journal of Forensic Medicine & Toxicology*.
- [38] Kushari, B. and Septiadi, L., 2022. A learning outcome assessment information system to facilitate Outcome-Based Education (OBE) Implementation. *Jurnal Pendidikan Teknologi dan Kejuruan*, 28(2), pp.238-250.
- [39] Othman, N., Saat, E.H.M., Muhamad, N.A., Anuar, N.H.K. and Majid, M.A., 2024. The Development an Outcome Base Education (OBE) System for Measuring Student Programme Attainment. *Journal of Computing Research and Innovation*, 9(2), pp.177-187.
- [40] Othman, N., Saat, E.H.M., Muhamad, N.A., Anuar, N.H.K. and Majid, M.A., 2024. The Development an Outcome Base Education (OBE) System for Measuring Student Programme Attainment. *Journal of Computing Research and Innovation*, 9(2), pp.177-187.
- [41] S. Xia, Y. Gao and B. Zhang, "Retracted: Teaching Strategies of Biological Experiments in Middle Schools Based on VR Technology," in *IEEE Access*, doi: 10.1109/ACCESS.2020.3022525.
- [42] H. Ma et al., "Learning Early Warning Guided by Course Objective Achievement via Knowledge State and Learning State Modeling," in *IEEE Transactions on Computational Social Systems*, doi: 10.1109/TCSS.2026.3656755.

- [43] M. Xie, X. Yu, X. Song, X. Dang and B. -H. Soong, "Joint Optimal Bilinear Equalizer for Cell-Free Massive MIMO Systems With Multi-Antenna Users," in *IEEE Transactions on Vehicular Technology*, doi: 10.1109/TVT.2026.3686454.
- [44] H. Du, S. Chen, H. Niu and H. Wang, "Teaching Exploration of Project-Based Teaching Based on OBE-SC-CIPP Framework in Computer Practice Course," 2022 4th International Workshop on Artificial Intelligence and Education (WAIE), Xiamen, China, 2022, pp. 51-55, doi: 10.1109/WAIE57417.2022.00018.
- [45] A. Abbas, C. Bautista-Flores, S. Hosseini, M. Mahrishi, A. Ali and N. Eltaiba, "Correlation of Students' Academic Progress, Counseling and Tutoring with Graduating Attributes in Outcome-Based Education (OBE): A Pilot Study of an OBEXcel Teaching and Learning Outcomes Project," 2025 IEEE Frontiers in Education Conference (FIE), Nashville, TN, USA, 2025, pp. 1-5, doi: 10.1109/FIE63693.2025.11328701.
- [46] Shaheen, S. (2019). Theoretical perspectives and current challenges of OBE framework. *International Journal of Engineering Education*, 1(2), 122-129.
- [47] Macayan, J. V. (2017). Implementing outcome-based education (OBE) framework: Implications for assessment of students' performance. *Educational Measurement and Evaluation Review*, 8(1), 1-10.
- [48] Syeed, M. M., Shihavuddin, A. S. M., Uddin, M. F., Hasan, M., & Khan, R. H. (2022). Outcome based education (OBE): Defining the process and practice for engineering education. *Ieee Access*, 10, 119170-119192.
- [49] Kaliannan, M., & Chandran, S. D. (2012). Empowering students through outcome-based education (OBE). *Research in Education*, 87(1), 50-63.
- [50] Kaliannan M, Chandran SD. Empowering students through outcome-based education (OBE). *Research in Education*. 2012 May;87(1):50-63.
- [51] Rajak, A., Shrivastava, A. K., & Shrivastava, D. P. (2019). Course outcome attainments in OBE for weak students. *International Journal of Innovative Technology and Exploring Engineering*, 8(11), 506-509.
- [52] Siregar, Z. A. B., Kadir, A. K., Muhibbudin, M. M., & Jannah, R. J. (2026). Analysis of Learning Assessment Suitability with Learning Outcomes in the Implementation of Outcome-Based Education (OBE) Curriculum. *JURNAL EDUSCIENCE*, 13(1), 51-63.
- [53] Syeed, M. M., Shihavuddin, A. S. M., Uddin, M. F., Hasan, M., & Khan, R. H. (2022). Outcome based education (OBE): Defining the process and practice for engineering education. *Ieee Access*, 10, 119170-119192.

- [54] Syeed, M.M., Shihavuddin, A.S.M., Uddin, M.F., Hasan, M. and Khan, R.H., 2022. Outcome based education (OBE): Defining the process and practice for engineering education. *Ieee Access*, 10, pp.119170-119192.
- [55] Kuikka, E., & Penttonen, M. (1995). Transformation of structured documents. University of Waterloo, Computer Science Department.
- [56] Rajski, J., & Cox, H. (1990, September). A method to calculate necessary assignments in algorithmic test pattern generation. In *Proceedings. International Test Conference 1990* (pp. 25-34). IEEE.
- [57] M. A. Chowdhury, K. K. S. Chisty, H. Tushar, K. M. F. Ahmed, and S. A. Waliullah, "Automating assessment and evaluation for a bachelor's degree program," *International Journal of Evaluation and Research in Education (IJERE)*, vol. 12, no. 4, pp. 2037– 2044, Dec. 2023, <http://doi.org/10.11591/ijere.v12i4.25479>
- [58] Shaheen, M. (2023). Decision tree for PLOs of undergraduate computing program based on CLO of computer programming. *Interactive Learning Environments*, 31(4), 2452-2470. <https://doi.org/10.1080/10494820.2021.1890621>
- [59] Chen, J., Lu, H., Zhou, H., & Zhou, Y. (2019, August). Exploration on curriculum teaching based on OBE and AI. In *2019 10th International Conference on Information Technology in Medicine and Education (ITME)* (pp. 385-389). IEEE. [10.1109/ITME.2019.00093](https://doi.org/10.1109/ITME.2019.00093)
- [60] Abu Zohair, L. M. (2019). Prediction of Student's performance by modelling small dataset size. *International Journal of Educational Technology in Higher Education*. <https://doi.org/10.1186/s41239-019-0160-3>
- [61] Akber, s & Roy, Ashutosh & Saha, Professor Gopal. (2023). IMPACT OF OUTCOME-BASED EDUCATION (OBE) ON LEARNERS' PERFORMANCE IN BUSINESS COURSES. *International Journal of Professional Business Review*. <https://doi.org/10.26668/BUSINESSREVIEW/2023.V8I8.2394>
- [62] N. Zaki, S. Turaev, K. Shuaib, A. Krishnan, and E. Mohamed, "Automating the mapping of s to using for accurate educational program evaluation," *Education and Information Technologies*, vol. 28, pp. 16723–16742, May 2023, <https://doi.org/10.1007/s10639-023-11877->
- [63] Arafah, S. (2015). Curriculum mapping in higher education: a case study and proposed content scope and sequence mapping tool. *Journal of Further and Higher Education*, 40(5), 585 611. <https://doi.org/10.1080/0309877X.2014.1000278>.
- [64] Jayalath, Vimukthini & Barthakur, Abhinava & Dawson, Shane & Tingey, Joanne & Crase, Lin & Kovanovic, Vitomir. (2025). Scaling Curriculum Mapping in Higher Education: Evaluating Generative AI's Role in Curriculum Analytics. https://doi.org/10.1007/978-3-031-98414-3_21

- [65] N. Zaki, S. Turaev, K. Shuaib, A. Krishnan, and E. Mohamed, "Automating the mapping of s to using for accurate educational program evaluation," *Education and Information Technologies*, vol. 28, pp. 16723–16742, May 2023, <https://doi.org/10.1007/s10639-023-11877->
- [66] Okoye, K., Arrona-Palacios, A., Camacho-Zuñiga, C. *et al.* Towards teaching analytics: a contextual model for analysis of students' evaluation of teaching through text mining and machine learning classification. *Educ Inf Technol* **27**, 3891–3933 (2022). <https://doi.org/10.1007/s10639-021-10751-5>
- [67] V. R. Tewari and M. N. Tiwari, "Impact assessment of outcome based approach in engineering education in India," *Procedia Computer Science*, vol. 172, pp. 791–796, 2020, doi: 10.1016/j.procs.2020.05.113. <https://www.sciencedirect.com/science/article/pii/S1877050920314423>
- [68] B. S. BLOOM, M. D. ENGELHART, E. J. FURST, W. H. HILL, AND D. R. KRATHWOHL, *TAXONOMY OF EDUCATIONAL OBJECTIVES: THE CLASSIFICATION OF EDUCATIONAL GOALS, HANDBOOK I: COGNITIVE DOMAIN*. NEW YORK, NY, USA: LONGMANS, GREEN, 1956. [10.4236/ce.2014.518180](https://doi.org/10.4236/ce.2014.518180)
- [69] ELALLAM, M., ET AL., "AUTOMATED EDUCATIONAL QUESTION GENERATION AT DIFFERENT BLOOM'S SKILL LEVELS USING LARGE LANGUAGE MODELS: STRATEGIES AND EVALUATION," *ARXIV PREPRINT ARXIV:2408.04394*, AUG. 2024. <https://arxiv.org/abs/2408.04394>
- [70] ES, J. JAMES, L. ESPINOSA ANKE, AND S. SCHOCKAERT, "RAGAS: AUTOMATED EVALUATION OF RETRIEVAL AUGMENTED GENERATION," IN *PROCEEDINGS OF THE 18TH CONFERENCE OF THE EUROPEAN CHAPTER OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS (EACL): SYSTEM DEMONSTRATIONS*, PP. 150–158, MAR. 2024. <https://aclanthology.org/2024.eacl-demo.16/>
- [71] M. Imran and N. Almusharraf, "Google Gemini as a Next Generation AI Educational Tool: A Review of Emerging Educational Technology," *Smart Learning Environments*, Springer, 2024. <https://link.springer.com/article/10.1186/s40561-024-00310-z>
- [72] Ouyang, F., Wu, M., Zheng, L., Zhang, L., and Jiao, P., "Integration of Artificial Intelligence Performance Prediction and Learning Analytics to Improve Student Learning in Online Engineering Course," *International Journal of Educational Technology in Higher Education*, vol. 20, no. 4, Springer, 2023. <https://doi.org/10.1186/s41239-022-00372-4>
- [73] P. LEWIS, E. PEREZ, A. PIKTUS, F. PETRONI, V. KARPUKHIN, N. GOYAL, H. KÜTTLER, M. LEWIS, W. YIH, T. ROCKTÄSCHEL, S. RIEDEL, AND D. KIELA, "RETRIEVAL-AUGMENTED GENERATION FOR KNOWLEDGE-INTENSIVE NLP TASKS," IN *ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NEURIPS)*, VOL. 33, PP. 9459–9474, 2020. <https://arxiv.org/abs/2005.11401>

- [74] ROJAS, M.P., ET AL., "AUTOMATED TEST CREATION USING LARGE LANGUAGE MODELS: A PRACTICAL APPLICATION," *APPLIED SCIENCES*, VOL. 14, NO. 19, P. 9125, MDPI, 2024. <https://www.mdpi.com/2076-3417/14/19/9125>
- [75] Google, "Gemini API Reference," Google AI for Developers. <https://ai.google.dev/api>
- [76] Render, "Web Services," Render Documentation. <https://render.com/docs/web-services>
- [77] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459-9474. <https://arxiv.org/abs/2005.11401>